

Chapter 28. Advanced SQL, Performance & Data Modeling

Recursive queries, query plans and indexes, normalization, star schemas with history, and safe schema changes, on Riverstone data.

Draft chapter v1.1

Contents

Chapter 28. Advanced SQL, Performance & Data Modeling

Why this matters

In plain English

28.1 Three practice databases

28.2 Recursive CTEs: walking trees of any depth

28.3 Advanced window frames

28.4 How a database finds rows

28.5 Reading query plans with EXPLAIN

28.6 Indexes in practice

28.7 Normalization, worked properly

28.8 The grain discipline

28.9 Dimensional modeling: facts, dimensions, and the star schema

28.10 Slowly changing dimensions

28.11 Denormalization trade-offs

28.12 Schema migrations: changing a live database safely

28.13 The same SQL in MySQL

Common mistakes and how to spot them

In the real world: the dashboard that got slower every month

Tools

The project: a star schema with history

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 28. Advanced SQL, Performance & Data Modeling

Part III — Advanced Analytics & Analytics Engineering

Chapter at a glance

You will learn to: walk org charts and bills of materials of any depth with recursive CTEs · protect a recursive query against loops · choose between `ROWS`, `RANGE`, and `GROUPS` frames, and use `EXCLUDE` and named windows · read a query plan with `EXPLAIN` and `EXPLAIN ANALYZE` · decide which indexes to create (single-column, composite, covering, partial) and which to avoid · recognize queries that can't use an index, and rewrite them · normalize a messy table to first, second, and third normal form · declare and test the grain of any table · design a star schema with facts, dimensions, and a date dimension · build slowly changing dimensions of types 1, 2, and 3 · decide when denormalizing is worth it · change a production database safely with versioned migration scripts.

Before you start: Chapter 12 (joins, keys, `CREATE TABLE`, `ALTER TABLE`, transactions, the fan-out trap) and Chapter 13 (CTEs, views, window functions, the `sales_lines` view).

Time needed: 18–24 hours of reading and practice, spread over three to four weeks.

Tools: PostgreSQL 16 or later and DBeaver, as in Chapter 12. MySQL 8.0 or later works for most of the chapter; section 28.13 shows what changes.

Practice data: the one-year database `riverstone_2025` with three new companion tables, a large generated database `riverstone_perf` (1.9 million order lines), and the `riverstone_lab` database you build yourself (section 28.1). Every query was run, and every result shown is the real output. Timings were measured on a small machine and are labeled with it.

Why this matters

The queries in Chapters 12 and 13 answer questions. This chapter is about what happens when those queries become part of how a company runs:

- "Show me everyone who reports to the Head of Production, however many levels down." A plain join handles one level. Real org charts, product structures, and account hierarchies have as many levels as they like.
- "The dashboard took two seconds last year. Now it takes forty." The query didn't change. The data grew, and nobody looked at how the database was finding the rows.
- "Why does the sales report show Sharma Hardware in two cities?" Somebody stored the same fact in two places, and updated only one of them.
- "Revenue by segment for last year changed, and nobody touched last year's orders." A customer was reclassified, and the report quietly rewrote history.

- "Can you add a column to the orders table this afternoon?" On a live system, with reports and applications reading that table, the honest answer is: yes, with a reviewed script, in the right order, and a way back.

These are the problems that separate someone who writes SQL from someone who is trusted with a company's data. They are also the topics that turn up in interviews for senior analyst, analytics engineer, and data engineer roles: "How would you speed up this query?", "What is the grain of this table?", "Explain a type 2 slowly changing dimension." You'll be able to answer all three with examples you've built yourself.

In plain English

A database is a large library, and your query is a request at the front desk.

Some requests are about **family trees**: "find this book, then every book that cites it, then every book that cites those." The librarian can't answer in one trip. She fetches the first shelf, reads it, and goes back for the next, until a trip brings back nothing new. That's a **recursive query**.

For every request, the librarian makes a **plan**. If you ask for one author's books, she can walk every aisle and look at every spine (a **scan**), or she can use the **catalog cards**, sorted by author, which tell her exactly which shelves to visit (an **index**). The catalog makes finding faster, but every new book now means writing a card too, so a library with a catalog for every possible question would spend all day filing cards. You can ask her to **explain her plan** before she sets off.

How the books are organized decides how hard the library is to keep correct. If each book's publisher address is written inside every book, a publisher that moves means correcting thousands of books. So you keep a publisher list and write only the publisher's name in each book. That's **normalization**: every fact in one place.

The reading room upstairs is organized differently. Readers there want to count and compare, not edit, so the librarian keeps a big **ledger of loans** (one line per loan: a **fact**) surrounded by small, friendly lookup lists: dates, readers, books (the **dimensions**). When a reader moves house, the lists keep the old address for old loans and the new address for new ones, so last year's report still describes last year. That's a **slowly changing dimension**.

And when the library wants a new shelf layout, it doesn't move books at random on a busy afternoon. It writes a numbered plan, gets it checked, and carries it out in order, noting each step in a logbook. That's a **migration**.

Keep the library in mind. Recursive queries, plans, indexes, normalization, dimensional models, and migrations are all in this chapter, and each one is a way of keeping a very large library fast and correct.

28.1 Three practice databases

This chapter uses three databases. Each section says which one it uses.

The one-year database (`riverstone_2025`) is Chapter 13's: the 24 key accounts, 175 orders in 2025, monthly targets, and leads, with the `sales_lines` view. (It is one slice of Riverstone, not the whole company: the key accounts the Chapter 13 team looks after.) For this chapter you add three tables to it by running `ch28_2025_addons.sql` from the companion files (`ch28_2025_addons_mysql.sql` for MySQL):

Table	One row is	Where it comes from	Used in
<code>staff</code>	one person on Riverstone's payroll, with their manager	the HR system (HRMS)	section 28.2
<code>parts</code> and <code>bom_lines</code>	a part (product, assembly, or bought material); one parent-and-child link in a product's structure	production	section 28.2
<code>customer_changes</code>	one change to a customer's city or segment during 2025	the ERP's audit log	sections 28.9–28.10

The `staff` table is Riverstone's reporting lines, from the managing director, Arvind Kapoor, down to the machine operators at the Taloja and Chakan plants: 35 people, including the key accounts sales team and the three regional sales managers. The five sales people you know from the `employees` table are in it too, with their `employee_id` filled in, so you can connect the two systems. (In the ERP's `employees` table Anita Rao has no manager, because the ERP only knows the sales team. In the HR system she reports to the managing director. Both are true; they're different systems with different scopes.)

The large database (`riverstone_perf`) exists to make performance visible. Indexes don't matter on 175 orders: every query is instant. So the companion script `generate_riverstone_perf.py` builds three years of Riverstone-like sales (2023 to 2025) with the same tables and columns as `riverstone_2025` :

Table	Rows
<code>customers</code>	5,000
<code>products</code>	8
<code>employees</code>	5
<code>orders</code>	714,285
<code>order_items</code>	1,926,847

It uses a fixed seed (28), so your data is identical to the book's. If you have the book's full three-year dataset loaded, every command in sections 28.4 to 28.6 works on that instead; only the timings change. The script writes CSV files and two load scripts, one for PostgreSQL and one for MySQL; the header of each file shows how to run it. The data takes about 200 MB in PostgreSQL. Apart from primary keys, the tables start with **no indexes**, because adding them is the lesson.

Simplification note. The loaded database has one setting changed: `ALTER DATABASE riverstone_perf SET max_parallel_workers_per_gather = 0;`. It turns off parallel query, which PostgreSQL uses on machines with several processor cores. Parallel plans are correct but longer to read, and the timings in this chapter were measured on a single-core machine where parallelism only adds noise. Leave the setting on your own copy if you like; the lessons are the same.

The lab database (`riverstone_lab`) is a scratch database you create and change freely, as in section 12.13. Sections 28.7, 28.10, and 28.12 build tables in it step by step. If you still have Chapter 12's lab, the first lab step drops and recreates it.

Watch out: timings are measurements, not facts. Every timing in this chapter was measured on the author's test machine: a single virtual processor with 3 GB of memory, running PostgreSQL 16 and MySQL 8.0.46 on Ubuntu 24.04. Each query was run seven times and the median is reported. Your laptop will give different numbers. What should match is the *plan* the database chooses and the *ratio* between a slow and a fast version. The companion script `ch28_perf.py` reruns every experiment.

28.2 Recursive CTEs: walking trees of any depth

Chapter 13 used `WITH RECURSIVE` once, to generate a list of months (section 13.8). Its real power is walking **hierarchies**: data where rows point at other rows of the same table.

The problem

"List everyone at Riverstone with their level in the organization."

Each `staff` row stores only its own manager:

```
SELECT staff_id, staff_name, job_title, manager_id
FROM staff
WHERE staff_name IN ('Arvind Kapoor', 'Harpreet Sethi', 'Ramesh Patil', 'Ajay Kumar', 'Gopal Sahu')
ORDER BY staff_id;
```

staff_id	staff_name	job_title	manager_id
100	Arvind Kapoor	Managing Director	
130	Harpreet Sethi	Head of Production	100
131	Ramesh Patil	Plant Manager, Taloja	130
133	Ajay Kumar	Shift Supervisor	131
136	Gopal Sahu	Machine Operator	133

(5 rows)

A self-join (section 12.10) gets you one level: each person and their manager. Two self-joins get you two levels. But the org chart is five levels deep today and may be six next year, and you can't write a query with a join for every level that might exist. You need a query that keeps going until it runs out of

people.

The recursive CTE

```
WITH RECURSIVE org AS (  
  -- anchor: the person with no manager  
  SELECT staff_id, staff_name, job_title, manager_id,  
         1 AS level  
  FROM staff  
  WHERE manager_id IS NULL  
  UNION ALL  
  -- recursive part: everyone whose manager is already in org  
  SELECT s.staff_id, s.staff_name, s.job_title, s.manager_id,  
         o.level + 1  
  FROM staff AS s  
  JOIN org AS o ON s.manager_id = o.staff_id  
)  
SELECT level, staff_name, job_title  
FROM org  
ORDER BY level, staff_id;
```

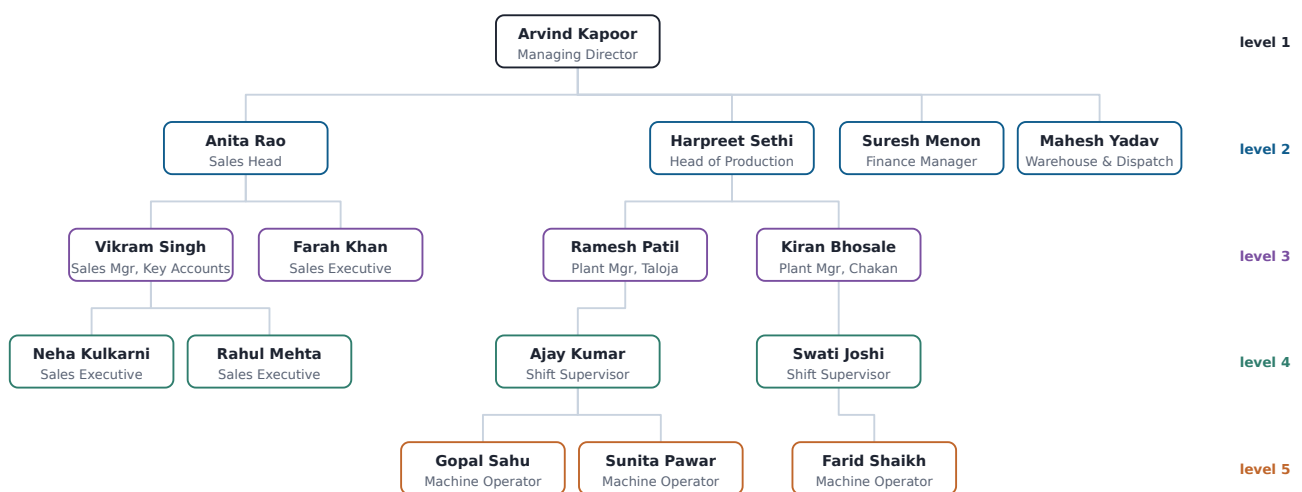
level	staff_name	job_title
1	Arvind Kapoor	Managing Director
2	Anita Rao	Sales Head
2	Suresh Menon	Finance Manager
2	Harpreet Sethi	Head of Production
2	Joseph D'Souza	Purchasing Manager
2	Mahesh Yadav	Warehouse & Dispatch Manager
2	Lakshmi Reddy	HR Manager
2	Zoya Mirza	Marketing Manager
2	Tenzin Dorji	Customer Support Lead
3	Vikram Singh	Sales Manager, Key Accounts
3	Farah Khan	Sales Executive
3	Meera Iyer	Sales Coordinator
3	Arjun Nair	Regional Sales Manager, South
3	Pooja Desai	Regional Sales Manager, West
3	Sandeep Gill	Regional Sales Manager, North
3	Priya Nambiar	Accounts Executive
3	Ramesh Patil	Plant Manager, Taloja
3	Kiran Bhosale	Plant Manager, Chakan
3	Mohan Das	Dispatch Supervisor
4	Neha Kulkarni	Sales Executive
4	Rahul Mehta	Sales Executive
4	Ajay Kumar	Shift Supervisor
4	Farhan Ali	Quality Inspector
4	Swati Joshi	Shift Supervisor
4	Divya Krishnan	Sales Executive
4	Vivek Chandran	Sales Executive
4	Sneha Pillai	Sales Executive
4	Nisha Bhatt	Sales Executive
4	Aditya Verma	Sales Executive
4	Rohit Kamat	Sales Executive
4	Karan Ahuja	Sales Executive
4	Ritu Bansal	Sales Executive
5	Gopal Sahu	Machine Operator
5	Sunita Pawar	Machine Operator
5	Farid Shaikh	Machine Operator

(35 rows)

How it works:

- A recursive CTE has two queries joined by `UNION ALL`. The first is the **anchor**: it runs once and produces the starting rows. Here that's Arvind Kapoor, the only person with no manager.
- The second is the **recursive part**. It refers to `org`, the CTE's own name. On each pass it sees only the rows the *previous* pass produced, and joins `staff` to them: "who reports to someone we found last time?" Pass 1 finds the eight department heads, pass 2 finds the ten managers below them, and so on.
- `o.level + 1` carries information down the tree. Each person's level is their manager's level plus one.
- The recursion stops by itself when a pass finds no new rows. After the machine operators, nobody reports to anybody new, so pass 5 returns nothing and the query ends.
- The final `SELECT` reads the whole accumulated result like any other CTE.

staff: each row points to its manager (manager_id)



The anchor row is the person with no manager (level 1). Each pass of the recursive part adds the next level down, and the query stops when a pass finds nobody new. The chart shows 16 of the 35 people (the regional sales team is left out); the query returns all of them.

Figure 28.1 – Each pass of the recursive part adds one level. The query stops when a pass finds nobody new.

Check a number by hand: Farah Khan reports directly to Anita Rao, who reports to Arvind Kapoor. Level 1, 2, 3. ✓ And the row count, 35, matches `SELECT COUNT(*) FROM staff`, so nobody was lost. That check matters more than it looks: a person whose `manager_id` points to someone who doesn't exist would silently disappear from a recursive walk. Exercise 4 asks you to find such orphans.

Walking up instead of down

"Who is in Gopal Sahu's chain of command?" Reverse the join: start from one person and repeatedly fetch *their* manager.

```

WITH RECURSIVE chain AS (
    SELECT staff_id, staff_name, manager_id, 0 AS steps_up
    FROM staff
    WHERE staff_name = 'Gopal Sahu'
    UNION ALL
    SELECT m.staff_id, m.staff_name, m.manager_id, c.steps_up + 1
    FROM staff AS m
    JOIN chain AS c ON m.staff_id = c.manager_id
)
SELECT steps_up, staff_name
FROM chain
ORDER BY steps_up;

```

```

steps_up |  staff_name
-----+-----
        0 | Gopal Sahu
        1 | Ajay Kumar
        2 | Ramesh Patil
        3 | Harpreet Sethi
        4 | Arvind Kapoor
(5 rows)

```

The only change is the join direction: `m.staff_id = c.manager_id` ("the person whose id is my manager's id") instead of `s.manager_id = o.staff_id`. An approval workflow uses exactly this walk. Riverstone's discount rules (Chapter 3) send large discounts up the chain until they reach someone with enough authority.

Counting everyone below each manager

"How many people does each manager have under them, directly or indirectly?" The trick is to pair every manager with every person below them. Start each person as "under themselves", then walk down:

```

WITH RECURSIVE under AS (
    SELECT staff_id AS manager_id, staff_id AS member_id
    FROM staff
    UNION ALL
    SELECT u.manager_id, s.staff_id
    FROM under AS u
    JOIN staff AS s ON s.manager_id = u.member_id
)
SELECT m.staff_name,
       m.job_title,
       COUNT(*) - 1 AS people_below
FROM under AS u
JOIN staff AS m ON m.staff_id = u.manager_id
GROUP BY m.staff_id, m.staff_name, m.job_title
HAVING COUNT(*) > 1
ORDER BY people_below DESC, m.staff_id;

```

staff_name	job_title	people_below
Arvind Kapoor	Managing Director	34
Anita Rao	Sales Head	16
Harpreet Sethi	Head of Production	8
Ramesh Patil	Plant Manager, Taloja	4
Arjun Nair	Regional Sales Manager, South	3
Pooja Desai	Regional Sales Manager, West	3
Vikram Singh	Sales Manager, Key Accounts	2
Sandeep Gill	Regional Sales Manager, North	2
Kiran Bhosale	Plant Manager, Chakan	2
Ajay Kumar	Shift Supervisor	2
Suresh Menon	Finance Manager	1
Swati Joshi	Shift Supervisor	1
Maresh Yadav	Warehouse & Dispatch Manager	1

(13 rows)

How it works:

- The anchor has one row per person: `(manager_id = me, member_id = me)` . That's 35 rows.
- Each pass takes every pair and adds the member's direct reports as new members of the same manager. Arvind's pairs grow to include everyone; Ajay Kumar's grow to include his two operators.
- `COUNT(*) - 1` removes the "under myself" row. `HAVING COUNT(*) > 1` keeps only people who manage somebody.
- Reconcile: the managing director has 34 people below him, which is everyone except himself (35 – 1). ✓ Harpreet Sethi's 8 is the two plant managers, two supervisors, one inspector, and three operators.

Combining a hierarchy with facts

Now join the tree to numbers. "What did Anita Rao's key accounts team sell in 2025?" Anita's branch of the tree now includes the regional sales managers and their executives, but the one-year database holds only the key accounts, so the query keeps the five people the ERP knows by their `employee_id` .

```
WITH RECURSIVE sales_team AS (
  SELECT staff_id, staff_name, employee_id
  FROM staff
  WHERE staff_name = 'Anita Rao'
  UNION ALL
  SELECT s.staff_id, s.staff_name, s.employee_id
  FROM staff AS s
  JOIN sales_team AS t ON s.manager_id = t.staff_id
)
SELECT t.staff_name,
       COUNT(DISTINCT sl.order_id) AS orders,
       ROUND(SUM(sl.net_revenue)) AS net_revenue
FROM sales_team AS t
LEFT JOIN sales_lines AS sl ON sl.sales_rep_id = t.employee_id
WHERE t.employee_id IS NOT NULL      -- the ERP knows only the key accounts team
GROUP BY t.staff_name
ORDER BY net_revenue DESC NULLS LAST;
```

staff_name	orders	net_revenue
Rahul Mehta	53	1494001
Farah Khan	64	1489073
Neha Kulkarni	46	1147895
Anita Rao	0	
Vikram Singh	0	

(5 rows)

The three executives' revenue adds up to ₹4,130,969. Chapter 13's total for 2025 is ₹4,335,471, so ₹204,502 belongs to orders with no sales rep recorded (Chapter 13's data-quality check found 11 of them). The team walk is right; the gap is a data problem, and it's exactly the kind of gap you should explain before anyone else finds it. Meera Iyer isn't in the result at all: she has no `employee_id`, because a coordinator doesn't take orders. Vikram Singh has no orders recorded under his own name in 2025; his team's orders are credited to the executives who took them.

Watch out: a recursive CTE's column types must match. In the next example, if the anchor returns `quantity` as `NUMERIC(8,3)` and the recursive part multiplies it (giving plain `NUMERIC`), PostgreSQL stops with `ERROR: recursive query "explode" column 3 has type numeric(8,3) in non-recursive term but type numeric overall`. The fix is to cast the anchor's column to the wider type, `CAST(b.quantity AS NUMERIC)`. MySQL has the opposite problem: it sizes text columns from the anchor, so a path string that grows on each pass fails with a `Data too long error` unless you `CAST` it to a long enough `CHAR` in the anchor (section 28.13).

Bills of materials

Riverstone's production team keeps a **bill of materials** (BOM): the list of parts that make each product, where some parts are themselves made of parts. The `bom_lines` table stores one link per row: "one unit of *parent* needs *quantity* units of *child*". The `parts` table says what each part is:

```
SELECT part_type, COUNT(*) AS parts
FROM parts
GROUP BY part_type
ORDER BY part_type;
```

part_type	parts
Assembly	9
Material	6
Product	5

(3 rows)

Five products, nine assemblies made in-house (a box body, a lid, a steel frame), and six materials bought from the suppliers you met in Chapter 12's lab.

"What does it take to make one Garden Chair?" This is called **exploding** the BOM. It's the org-chart walk with one addition: quantities multiply on the way down.

```

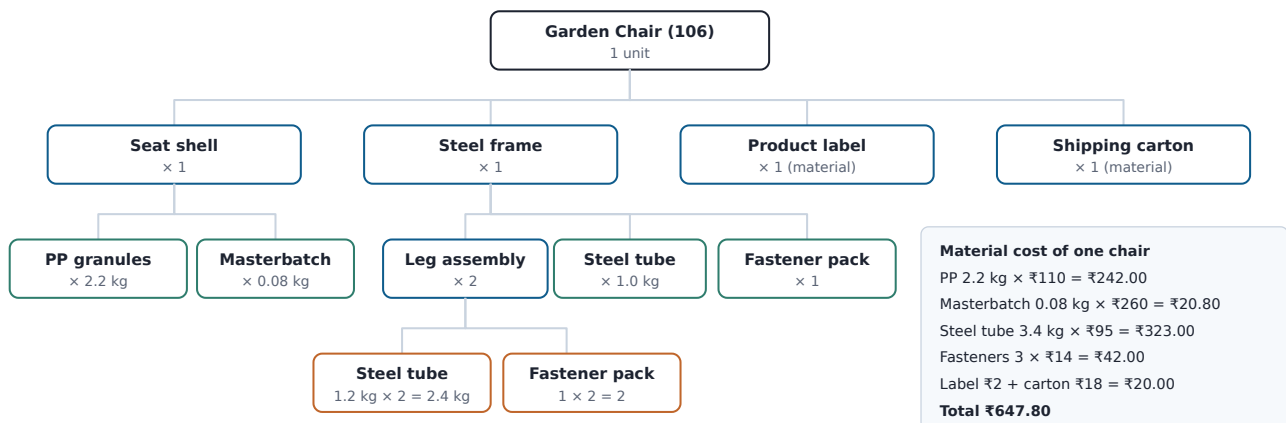
WITH RECURSIVE explode AS (
  SELECT b.parent_part_id AS product_id,
         b.child_part_id,
         CAST(b.quantity AS NUMERIC) AS qty_per_product,
         1 AS depth
  FROM bom_lines AS b
  WHERE b.parent_part_id = 106
  UNION ALL
  SELECT e.product_id,
         b.child_part_id,
         e.qty_per_product * b.quantity,
         e.depth + 1
  FROM explode AS e
  JOIN bom_lines AS b ON b.parent_part_id = e.child_part_id
)
SELECT e.depth,
       p.part_name,
       p.part_type,
       CAST(e.qty_per_product AS NUMERIC(8,3)) AS qty_per_chair,
       p.unit
FROM explode AS e
JOIN parts AS p ON p.part_id = e.child_part_id
ORDER BY e.depth, p.part_id;

```

depth	part_name	part_type	qty_per_chair	unit
1	Seat shell	Assembly	1.000	each
1	Steel frame	Assembly	1.000	each
1	Product label	Material	1.000	each
1	Shipping carton	Material	1.000	each
2	Leg assembly	Assembly	2.000	each
2	Polypropylene granules	Material	2.200	kg
2	Color masterbatch	Material	0.080	kg
2	Steel tube	Material	1.000	kg
2	Fastener pack	Material	1.000	each
3	Steel tube	Material	2.400	kg
3	Fastener pack	Material	2.000	each

(11 rows)

Bill of materials: quantities multiply as you go down



Orange boxes are found on the third pass: the recursive CTE multiplies each child's quantity by its parent's running quantity.

Figure 28.2 — Quantities multiply down the tree: 2 leg assemblies × 1.2 kg each = 2.4 kg of steel tube.

How it works:

- The anchor fetches the chair's direct children (depth 1).

- The recursive part fetches each child's own children and multiplies: the frame needs 2 leg assemblies, each leg assembly needs 1.2 kg of steel tube, so through the legs one chair needs $2 \times 1.2 = 2.4$ kg. That's the orange row at depth 3.
- Steel tube appears twice (1.0 kg directly in the frame, 2.4 kg in the legs). That's correct: it's used in two places. To buy steel, you add them: 3.4 kg per chair.
- Assemblies appear in the list, but you don't buy them. For purchasing, keep only `part_type = 'Material'`.

Rolling up material cost

"What are the materials in each product worth, compared with its standard cost?" Explode every product at once, keep the materials, multiply by their cost, and add up per product:

```
WITH RECURSIVE explode AS (
  SELECT b.parent_part_id AS product_id,
         b.child_part_id,
         CAST(b.quantity AS NUMERIC) AS qty_per_product
  FROM bom_lines AS b
  WHERE b.parent_part_id IN (SELECT part_id FROM parts WHERE part_type = 'Product')
  UNION ALL
  SELECT e.product_id,
         e.child_part_id,
         e.qty_per_product * b.quantity
  FROM explode AS e
  JOIN bom_lines AS b ON b.parent_part_id = e.child_part_id
)
SELECT p.part_id    AS product_id,
       p.part_name  AS product_name,
       CAST(SUM(e.qty_per_product * m.material_cost) AS NUMERIC(10,2)) AS material_cost,
       pr.unit_cost AS standard_cost
FROM explode AS e
JOIN parts    AS m ON m.part_id = e.child_part_id AND m.part_type = 'Material'
JOIN parts    AS p ON p.part_id = e.product_id
JOIN products AS pr ON pr.product_id = p.part_id
GROUP BY p.part_id, p.part_name, pr.unit_cost
ORDER BY p.part_id;
```

product_id	product_name	material_cost	standard_cost
101	Storage Box 10L	110.30	300.00
102	Storage Box 25L	199.30	540.00
104	Food Container Set	59.42	430.00
106	Garden Chair	647.80	850.00
108	Stackable Bin	55.40	190.00

(5 rows)

Check the chair by hand, using the explosion above: polypropylene $2.2 \text{ kg} \times ₹110 = ₹242.00$, masterbatch $0.08 \text{ kg} \times ₹260 = ₹20.80$, steel tube $3.4 \text{ kg} \times ₹95 = ₹323.00$, fasteners $3 \times ₹14 = ₹42.00$, label ₹2.00, carton ₹18.00. Total ₹647.80. ✓

What to tell the production head. Materials are three-quarters of the Garden Chair's ₹850 standard cost (₹647.80, or 76.2%), and half of that (₹323.00, 49.9%) is steel tube. A 10% rise in the steel price would add ₹32.30 to every chair, while the same rise in polypropylene would add ₹24.20. For the Food

Container Set, materials are only ₹59.42 of ₹430: the cost is in molding time, not plastic. Those are two different conversations with two different suppliers. (Standard cost also includes labor, machine time, and overheads, which the BOM doesn't hold.)

Where is a part used?

The reverse question matters when a supplier has a problem: *"Kaveri Steel Works can't deliver steel tube this month. Which products are affected, and how much tube does each unit need?"* Walk up from the part:

```
WITH RECURSIVE uses AS (
    SELECT b.parent_part_id, CAST(b.quantity AS NUMERIC) AS qty
    FROM bom_lines AS b
    WHERE b.child_part_id = 305
    UNION ALL
    SELECT b.parent_part_id, u.qty * b.quantity
    FROM uses AS u
    JOIN bom_lines AS b ON b.child_part_id = u.parent_part_id
)
SELECT p.part_name,
       CAST(SUM(u.qty) AS NUMERIC(8,3)) AS kg_steel_tube_per_unit
FROM uses AS u
JOIN parts AS p ON p.part_id = u.parent_part_id
WHERE p.part_type = 'Product'
GROUP BY p.part_name;
```

part_name	kg_steel_tube_per_unit
Garden Chair	3.400

(1 row)

Only the Garden Chair, at 3.4 kg per unit, matching the explosion. ✓ This is called a **where-used** query.

Guarding against loops

A tree has no loops: nobody is their own manager's manager. But data entered by people can contain one. Suppose, by mistake, person 1 reports to 3, 3 reports to 2, and 2 reports to 1. A recursive walk would go round forever. Here's what it produces, stopped after seven rows by `LIMIT`:

```
WITH RECURSIVE bad_staff (staff_id, manager_id) AS (
    VALUES (1, 3), (2, 1), (3, 2)
),
chain AS (
    SELECT staff_id, manager_id, 1 AS steps
    FROM bad_staff
    WHERE staff_id = 1
    UNION ALL
    SELECT b.staff_id, b.manager_id, c.steps + 1
    FROM bad_staff AS b
    JOIN chain AS c ON b.staff_id = c.manager_id
)
SELECT * FROM chain LIMIT 7;
```

staff_id	manager_id	steps
1	3	1
3	2	2
2	1	3
1	3	4
3	2	5
2	1	6
1	3	7

(7 rows)

Without the `LIMIT`, the query never ends (until it runs out of memory or someone cancels it). There are three defenses:

1. **A depth limit** in the recursive part: `WHERE c.steps < 20`. Simple, works everywhere, and an honest org chart never needs twenty levels.
2. **A visited-path check**: carry an array or string of the ids seen so far, and stop when the next id is already in it.
3. **PostgreSQL's `CYCLE` clause** (PostgreSQL 14 and later), which does the path check for you:

```
WITH RECURSIVE bad_staff (staff_id, manager_id) AS (
  VALUES (1, 3), (2, 1), (3, 2)
),
chain AS (
  SELECT staff_id, manager_id
  FROM bad_staff
  WHERE staff_id = 1
  UNION ALL
  SELECT b.staff_id, b.manager_id
  FROM bad_staff AS b
  JOIN chain AS c ON b.staff_id = c.manager_id
)
CYCLE staff_id SET is_loop USING visited
SELECT staff_id, manager_id, is_loop, visited
FROM chain;
```

staff_id	manager_id	is_loop	visited
1	3	f	{(1)}
3	2	f	{(1),(3)}
2	1	f	{(1),(3),(2)}
1	3	t	{(1),(3),(2),(1)}

(4 rows)

`CYCLE staff_id` watches that column. When a row's `staff_id` is already in the path, it's marked `is_loop = t` and not followed further. The `visited` column shows the path. Filter `WHERE is_loop` to list the loops so someone can fix the data.

Dialect note: ordering a tree. PostgreSQL also has a `SEARCH DEPTH FIRST BY` clause that numbers rows in the order you'd draw an indented org chart (each manager followed by their whole team). It's used in exercise 5's answer. MySQL has neither `SEARCH` nor `CYCLE`; section 28.13 builds a sort path by hand instead, which works in both databases.

28.3 Advanced window frames

Section 13.6 introduced frames and the trap of the default `RANGE` frame. This section completes the picture: the three frame types, frames measured in days, excluding rows, reusing window definitions, and the percentile functions.

ROWS, RANGE, and GROUPS

A frame answers "*which rows around me count?*", and there are three ways to measure "around":

Frame type	Counts in units of	"1 PRECEDING" means
ROWS	physical rows	the one row before me
GROUPS	groups of rows with the same <code>ORDER BY</code> value	every row in the previous distinct value (a <i>peer group</i>)
RANGE	the <code>ORDER BY</code> value itself	every row whose value is within 1 unit (1 day, ₹1...) of mine

The difference only shows when values repeat or have gaps. On 16, 17, and 20 February 2025, Riverstone received two orders each:

```
SELECT o.order_date,
       o.order_id,
       COUNT(*) OVER w_rows AS rows_in_frame,
       COUNT(*) OVER w_groups AS rows_in_2_dates,
       COUNT(*) OVER w_range AS rows_in_2_days
FROM orders AS o
WHERE o.order_date BETWEEN DATE '2025-02-14' AND DATE '2025-02-21'
WINDOW w_rows AS (ORDER BY o.order_date ROWS BETWEEN 1 PRECEDING AND CURRENT ROW),
       w_groups AS (ORDER BY o.order_date GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW),
       w_range AS (ORDER BY o.order_date RANGE BETWEEN INTERVAL '1 day' PRECEDING AND CURRENT
ROW)
ORDER BY o.order_date, o.order_id;
```

order_date	order_id	rows_in_frame	rows_in_2_dates	rows_in_2_days
2025-02-16	10011	1	2	2
2025-02-16	10012	2	2	2
2025-02-17	10013	2	4	4
2025-02-17	10014	2	4	4
2025-02-20	10015	2	4	2
2025-02-20	10016	2	4	2

(6 rows)

Reading it:

- ROWS** counts the row before and this row: always 2 after the first row, whatever the dates.
- GROUPS** counts this date's orders plus the previous *date that has orders*. On 20 February that's 20 Feb and 17 Feb: 4 orders, even though 17 Feb is three days earlier.
- RANGE ... INTERVAL '1 day'** counts orders from yesterday and today. On 20 February, 19 February had no orders, so only today's 2 count. On 17 February, 16 February is exactly one day earlier: 4.

Use `ROWS` for "the last N records", `GROUPS` for "the last N distinct dates that had activity", and `RANGE` with an interval for "the last N calendar days".

The **WINDOW clause** at the end of that query is also new. It names a window definition once (`w_rows`), so `OVER w_rows` can reuse it. When several columns share the same partition and order, a named window keeps the query readable and guarantees they really do use the same window. You can extend a named window too: `OVER (w ROWS BETWEEN ...)` adds a frame to a window that defines only the partition and order.

Calendar windows with `RANGE` and intervals

"What was revenue over the last seven days, on each day we took orders?" Riverstone doesn't take orders every day, so "the last seven rows" is not "the last seven days":

```
WITH daily AS (
  SELECT order_date, SUM(net_revenue) AS revenue
  FROM sales_lines
  WHERE order_date BETWEEN DATE '2025-11-20' AND DATE '2025-12-10'
  GROUP BY order_date
)
SELECT order_date,
  ROUND(revenue) AS revenue,
  ROUND(SUM(revenue) OVER (ORDER BY order_date
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)) AS last_3_rows,
  ROUND(SUM(revenue) OVER (ORDER BY order_date
    RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW)) AS last_7_days
FROM daily
ORDER BY order_date;
```

order_date	revenue	last_3_rows	last_7_days
2025-11-22	86615	86615	86615
2025-11-23	2731	89346	89346
2025-11-25	10213	99559	99559
2025-11-26	10060	23004	109619
2025-11-28	17200	37473	126819
2025-12-01	71862	99122	109334
2025-12-04	9809	98870	98870
2025-12-05	13705	95375	95375
2025-12-08	65818	89331	89331
2025-12-10	7698	87221	97029

(10 rows)

Hand-check 26 November. The last three rows are 23, 25, and 26 November: ₹2,731 + ₹10,213 + ₹10,060 = ₹23,004. ✓ The last seven days run from 20 to 26 November, which includes the big ₹86,615 order on the 22nd: ₹86,615 + ₹2,731 + ₹10,213 + ₹10,060 = ₹109,619. ✓ On 1 December, the seven days (25 November to 1 December) include 25, 26, and 28 November and 1 December: ₹109,334. ✓

This is Chapter 13's date-spine problem (Pattern 5) solved without a spine: `RANGE` with an interval measures by the calendar, so missing days take care of themselves. Because the frame begins before the query's first date, the first few rows here only see part of their window, which is why 22 November shows a single order. In a real report, start the `WHERE` six days earlier and filter the output afterwards.

Dialect note. PostgreSQL writes `INTERVAL '6 days'` ; MySQL writes `INTERVAL 6 DAY` . MySQL supports `RANGE` with intervals but not `GROUPS` (section 28.13). SQL Server supports neither interval ranges nor `GROUPS` ; there, use a date spine.

Excluding the current row

"Was this month unusual compared with the other months of its quarter?" Comparing a month with an average that includes itself dilutes the comparison. `EXCLUDE CURRENT ROW` leaves it out:

```
WITH monthly AS (  
    SELECT DATE_TRUNC('month', order_date)::date AS month,  
           SUM(net_revenue) AS revenue  
    FROM sales_lines  
    GROUP BY 1  
)  
SELECT month,  
       ROUND(revenue) AS revenue,  
       ROUND(AVG(revenue) OVER (  
           PARTITION BY DATE_TRUNC('quarter', month)  
           ORDER BY month  
           ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
           EXCLUDE CURRENT ROW)) AS avg_other_months_in_quarter  
FROM monthly  
WHERE month >= DATE '2025-07-01'  
ORDER BY month;
```

month	revenue	avg_other_months_in_quarter
2025-07-01	232692	443798
2025-08-01	329282	395504
2025-09-01	558315	280987
2025-10-01	681071	536616
2025-11-01	633408	560447
2025-12-01	439824	657239

(6 rows)

September's ₹558,315 against the other two months of Q3 (₹232,692 and ₹329,282, averaging ₹280,987) shows the festive season starting in September, not October. (Check: $(232,692 + 329,282) \div 2 = 280,987$. ✓) `EXCLUDE` has four forms: `CURRENT ROW` , `GROUP` (the current row and its peers), `TIES` (the peers but not the current row), and `NO OTHERS` (the default).

FIRST_VALUE, LAST_VALUE, and the default frame

Chapter 13's mistakes table warned that `LAST_VALUE` with the default frame returns the current row. Here it is on Patel Kitchenware's orders, with the fix beside it:

```

WITH order_values AS (
  SELECT o.customer_id, o.order_id, o.order_date, SUM(sl.net_revenue) AS order_value
  FROM orders AS o
  JOIN sales_lines AS sl ON sl.order_id = o.order_id
  WHERE o.customer_id = 2
  GROUP BY o.customer_id, o.order_id, o.order_date
)
SELECT order_date,
       ROUND(order_value) AS order_value,
       ROUND(FIRST_VALUE(order_value) OVER w) AS first_order,
       ROUND(LAST_VALUE(order_value) OVER w) AS last_value_default,
       ROUND(LAST_VALUE(order_value) OVER (w ROWS BETWEEN UNBOUNDED PRECEDING
                                           AND UNBOUNDED FOLLOWING)) AS latest_order
FROM order_values
WINDOW w AS (PARTITION BY customer_id ORDER BY order_date, order_id)
ORDER BY order_date;

```

order_date	order_value	first_order	last_value_default	latest_order
2025-01-02	2900	2900	2900	39870
2025-03-28	35138	2900	35138	39870
2025-04-11	19825	2900	19825	39870
2025-05-04	23650	2900	23650	39870
2025-06-10	14155	2900	14155	39870
2025-08-14	38610	2900	38610	39870
2025-11-09	39870	2900	39870	39870

(7 rows)

With an `ORDER BY` and no frame, the frame ends at the current row, so "the last value in the frame" is always the current row: `last_value_default` copies `order_value`. Extending the frame to `UNBOUNDED FOLLOWING` gives the real latest order, ₹39,870 on 9 November. `FIRST_VALUE` works with the default frame because the frame always starts at the first row. `NTH_VALUE(order_value, 2)` gives the second order, with the same frame caution.

Percentiles and quartiles

"What's a typical order worth?" Chapter 4 explained why the median is often more honest than the mean. SQL calculates percentiles with **ordered-set aggregates**, written with `WITHIN GROUP` :

```

WITH order_values AS (
  SELECT order_id, SUM(net_revenue) AS order_value
  FROM sales_lines
  GROUP BY order_id
)
SELECT COUNT(*) AS orders,
       ROUND(AVG(order_value)) AS mean_value,
       ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY order_value)::NUMERIC) AS median_value,
       ROUND(PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY order_value)::NUMERIC) AS p90_value
FROM order_values;

```

orders	mean_value	median_value	p90_value
173	25061	21375	48334

(1 row)

The mean (₹25,061) is above the median (₹21,375), because a few large orders pull it up. One order in ten is worth more than ₹48,334. `PERCENTILE_CONT` interpolates between two middle values when needed; `PERCENTILE_DISC` returns an actual value from the data. These are aggregates, not window functions: in PostgreSQL they can't take `OVER`.

To put every order into a band instead, use the window function `NTILE(n)`, which splits ordered rows into n groups as equal in size as possible:

```
WITH order_values AS (  
    SELECT order_id, SUM(net_revenue) AS order_value  
    FROM sales_lines  
    GROUP BY order_id  
)  
,  
banded AS (  
    SELECT order_value, NTILE(4) OVER (ORDER BY order_value) AS quartile  
    FROM order_values  
)  
SELECT quartile,  
       COUNT(*) AS orders,  
       ROUND(MIN(order_value)) AS smallest,  
       ROUND(MAX(order_value)) AS largest,  
       ROUND(SUM(order_value)) AS revenue  
FROM banded  
GROUP BY quartile  
ORDER BY quartile;
```

quartile	orders	smallest	largest	revenue
1	44	1378	12355	307338
2	43	12398	21375	716217
3	43	21650	35138	1196060
4	43	35300	100278	2115857

(4 rows)

173 orders don't divide by 4, so the first band gets the extra order ($44 + 43 + 43 + 43 = 173$ ✓). The revenue column tells the story: the largest quarter of orders brings ₹2,115,857, almost half of the year's ₹4,335,471 (48.8%), and the smallest quarter brings 7.1%. The bands add up to the year's total: $307,338 + 716,217 + 1,196,060 + 2,115,857 = 4,335,472$, one rupee more than the total because each band was rounded separately. ✓

Watch out: `NTILE` splits by count, not by value. If many orders have the same value, `NTILE` may put identical values into different bands, because it only cares about getting equal-sized groups. For value-based bands, compute the cut points with `PERCENTILE_CONT` and use `CASE`.

28.4 How a database finds rows

To speed up a query, you need a picture of what the database does with it. The picture is simpler than you might expect.

Tables live in pages

PostgreSQL stores a table as a file divided into **pages** (also called blocks) of 8 KB each. Rows are packed into pages in roughly the order they were inserted, not in any useful order. `riverstone_perf`'s `orders` table is 41 MB, which is about 5,200 pages; `order_items` is about 101 MB. Reading pages from disk (or even from memory) is most of the work a query does, so the question behind every performance problem is: **how many pages did the database have to read to find the rows it needed?**

Scanning

With no other help, the only way to find customer 2718's orders is a **sequential scan** (in PostgreSQL's plans, `Seq Scan`): read every page and check every row. For 714,285 orders, that's every one of them, to keep 40. It is slow when you want a few rows, but it isn't always wrong. To total *all* the year's revenue, reading everything is exactly the job, and a scan does it in the most efficient order.

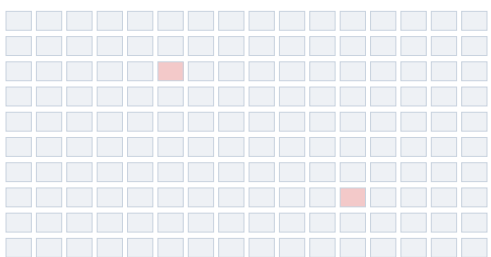
Indexes

An **index** is a separate structure that stores one column's values (or several columns') in sorted order, each with a pointer to where its row lives in the table. It works like the index at the back of this book: to find "grain", you don't read every page, you look it up and go straight to the right page.

The standard kind is the **B-tree** (balanced tree). Its top page, the root, divides the key range into a few hundred slices and points to a page for each; those pages divide their slices again, until the bottom **leaf** pages list individual values and row pointers. Because each page fans out into hundreds of children, even millions of values need only three or four levels. Finding customer 2718 means reading the root, a page or two below it, a leaf, and then the 40 table pages holding those rows.

Finding customer 2718's 40 orders among 714,285

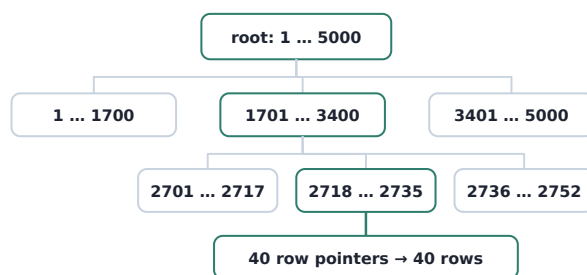
Sequential scan: read every row



714,285 rows checked, 714,245 thrown away

median 30.11 ms

B-tree index: follow the sorted keys



a few index pages, then 40 table pages (Heap Blocks: 40)

median 0.08 ms

PostgreSQL 16 on a 1-vCPU sandbox, riverstone_perf. Your times will differ; the ratio is what matters.

Figure 28.3 – The same question answered two ways. On the test machine the index made it about 380 times faster.

What indexes cost

Indexes aren't free, which is why databases don't index every column automatically:

- **Space.** The index on `orders.customer_id` takes 4,960 kB; the primary key index on `order_items` takes 41 MB, almost half the size of the table itself.
- **Slower writes.** Every `INSERT`, every `DELETE`, and every `UPDATE` of an indexed column must update each index too. Section 28.6 measures this.

- **Choices for the planner.** More indexes mean more plans to consider, and occasionally a poor choice.

A **primary key** always has an index, because the database needs one to enforce uniqueness. So does a **UNIQUE** constraint. Everything else is your decision.

Watch out: PostgreSQL does not index foreign keys for you. Declaring `order_items.order_id REFERENCES orders(order_id)` creates a rule, not an index. Joins on that column, and every delete from `orders` (which must check for lines that still point at the order), then scan the whole `order_items` table. MySQL's InnoDB engine *does* create an index for each foreign key automatically. Section 28.6 shows the difference on real data.

28.5 Reading query plans with EXPLAIN

Chapter 12 (section 12.11) promised you'd see what the database actually does with a query. Put **EXPLAIN** in front of any query, and PostgreSQL shows its **query plan**, the steps it has chosen, without running the query:

```
EXPLAIN
SELECT order_id, order_date, status
FROM orders
WHERE customer_id = 2718;
```

```
QUERY PLAN
-----
Seq Scan on orders (cost=0.00..14149.56 rows=98 width=17)
  Filter: (customer_id = 2718)
(2 rows)
```

(Run on `riverstone_perf` with no index on `customer_id`. The outputs in sections 28.5 and 28.6 come from the recorded runs in `ch28_perf_log.txt`, because timings change on every run.)

Reading it:

- **Seq Scan on orders** : the step, a sequential scan of the whole table.
- **Filter: (customer_id = 2718)** : the condition checked on each row as it's read.
- **cost=0.00..14149.56** : the planner's *estimate* of effort, in arbitrary units (roughly, the cost of reading one page in order is 1). The first number is the cost before the first row can be returned; the second is the cost to return all rows. Costs let the planner compare plans. They are not milliseconds.
- **rows=98** : the estimated number of rows returned. The planner guesses from **statistics** it keeps about each column (how many distinct values, which are most common), refreshed by **ANALYZE**. The real answer is 40, so the estimate is in the right neighborhood.
- **width=17** : the estimated average size of each row in bytes.

EXPLAIN ANALYZE: run it and measure

EXPLAIN ANALYZE runs the query for real and adds what actually happened. The option **COSTS OFF** hides the estimates to keep lines short:

```
EXPLAIN (ANALYZE, COSTS OFF)
SELECT order_id, order_date, status
FROM orders
WHERE customer_id = 2718;
```

QUERY PLAN

```
-----
Seq Scan on orders (actual time=0.610..29.350 rows=40 loops=1)
  Filter: (customer_id = 2718)
  Rows Removed by Filter: 714245
Planning Time: 0.151 ms
Execution Time: 29.381 ms
(5 rows)
```

- **actual time=0.610..29.350** : milliseconds until the first row, and until the last row.
- **rows=40 loops=1** : rows actually returned, and how many times this step ran.
- **Rows Removed by Filter: 714245** : the tell-tale sign. The database read 714,285 rows to keep 40. When you see a filter throwing away almost everything, an index on the filtered column is the first thing to consider.
- **Execution Time** : the whole query, 29.4 ms on this run. Across seven runs the median was 30.11 ms.

Watch out: EXPLAIN ANALYZE really runs the statement. `EXPLAIN ANALYZE DELETE FROM orders ...` deletes the rows. To measure a change safely, wrap it: `BEGIN; EXPLAIN ANALYZE ...; ROLLBACK; .`

Now create an index and ask again:

```
CREATE INDEX idx_orders_customer_id ON orders (customer_id);

EXPLAIN (ANALYZE, COSTS OFF)
SELECT order_id, order_date, status
FROM orders
WHERE customer_id = 2718;
```

QUERY PLAN

```
-----
Bitmap Heap Scan on orders (actual time=0.014..0.037 rows=40 loops=1)
  Recheck Cond: (customer_id = 2718)
  Heap Blocks: exact=40
   -> Bitmap Index Scan on idx_orders_customer_id (actual time=0.007..0.008 rows=40 loops=1)
        Index Cond: (customer_id = 2718)
Planning Time: 0.209 ms
Execution Time: 0.077 ms
(7 rows)
```

Building the index took under half a second. The query's median fell from 30.11 ms to 0.08 ms. **Read plans from the most indented line outwards**: the innermost step runs first and feeds the one above it.

- **Bitmap Index Scan** reads the index and collects the locations of the 40 matching rows (`Index Cond` is the condition the index answered).
- **Bitmap Heap Scan** then visits those locations in page order. **Heap Blocks: exact=40** says it read 40 table pages: this customer's orders are scattered, one per page. (The *heap* is PostgreSQL's name for the table's own pages.)

The planner chooses, and it can say no

PostgreSQL has several ways to use an index, and chooses by estimated cost:

Plan node	What it does	Typical when
Seq Scan	reads every page	most rows are needed, or no useful index
Index Scan	walks the index, fetching each row as it goes	few rows, or rows wanted in index order
Bitmap Index Scan + Bitmap Heap Scan	collects all matching locations first, then reads pages in order	a moderate number of rows
Index Only Scan	answers from the index alone, never touching the table	every column needed is in the index

Try the customer with the most orders, customer 4534, who has 23,526 of them. The same index is used, but the plan reads 5,170 pages and takes a median of 9.72 ms. And for a condition that matches almost everything, the planner ignores the index entirely, even when one exists:

```
EXPLAIN
SELECT order_id FROM orders WHERE status = 'Delivered';
```

QUERY PLAN

```
-----
Seq Scan on orders  (cost=0.00..14149.56 rows=688404 width=4)
  Filter: ((status)::text = 'Delivered'::text)
(2 rows)
```

689,050 of the 714,285 orders are delivered (96.5%). Jumping from an index to a table page 689,050 times would be slower than reading the table from start to end, and the planner knows it from its statistics. **An index helps when a condition is selective**, meaning it keeps a small fraction of rows.

Joins in a plan

Plans for joins have one more thing to read: the **join method**. Here's "customer 2718's total revenue", which joins orders to their lines, run while `order_items.order_id` still has no index:

Reading EXPLAIN ANALYZE: from the inside out (customer 2718's revenue, no index on order_items.order_id)

```
4 Aggregate (actual time=225.300..225.326 rows=1)
    -> Hash Join (actual time=4.120..225.209 rows=108)
        Hash Cond: (oi.order_id = o.order_id)
        3 -> Seq Scan on order_items oi (actual time=0.013..116.108 rows=1926847)
        2 -> Hash (actual time=0.060..0.084 rows=40)
            -> Bitmap Heap Scan on orders o (actual time=0.018..0.051 rows=40)
                1 -> Bitmap Index Scan on idx_orders_customer_id (rows=40)
Execution Time: 225.368 ms
```

- 1 Read the most indented step first: the index finds customer 2718's 40 orders in a fraction of a millisecond.
- 2 Their 40 order ids go into a small in-memory hash table.
- 3 The expensive step: every one of 1,926,847 order lines is read and checked against that hash table.
- 4 The top node is the final result. Its second time (225 ms) is the whole query; the Seq Scan's 116 ms is about half of it.

Actual times are in milliseconds, shown as first row..last row. The other 109 ms is the Hash Join checking each of those lines.

Figure 28.4 – Read the plan from the inside out, and look for the step where the time jumps.

PostgreSQL uses three join methods:

- **Nested loop:** for each row on one side, look up matches on the other. Excellent when one side is small *and* the other side has an index on the join column. Disastrous when the other side must be scanned each time.
- **Hash join:** load the smaller side into an in-memory hash table, then scan the larger side once, probing the table. Good for large, unindexed joins. That's what Figure 28.4 shows: the planner had no index to look lines up by `order_id`, so it scanned all 1.9 million lines to find 108.
- **Merge join:** sort both sides on the join key, then walk them together. Good when both sides are already sorted, for example by an index.

You don't choose the method; the planner does. Your job is to give it the indexes that make the cheap methods possible, and then check that it used them.

Try it. Run `EXPLAIN SELECT * FROM sales_lines; in riverstone_2025`. A view is replaced by its query before planning, so the plan shows the joins inside `sales_lines`. With tables this small, you'll see sequential scans and hash joins, and that's the right choice.

28.6 Indexes in practice

This section runs six experiments on `riverstone_perf`. Each one starts from a question, measures it, changes one thing, and measures again. The table at the end collects the results.

1. Index the foreign keys you join on

The query from Figure 28.4, run before and after one new index:

```
CREATE INDEX idx_order_items_order_id ON order_items (order_id);

EXPLAIN (ANALYZE, COSTS OFF)
SELECT SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)) AS net_revenue
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
WHERE o.customer_id = 2718;
```

QUERY PLAN

```
-----
Aggregate (actual time=0.242..0.243 rows=1 loops=1)
  -> Nested Loop (actual time=0.024..0.195 rows=108 loops=1)
    -> Bitmap Heap Scan on orders o (actual time=0.016..0.046 rows=40 loops=1)
        Recheck Cond: (customer_id = 2718)
        Heap Blocks: exact=40
    -> Bitmap Index Scan on idx_orders_customer_id (actual time=0.009..0.009 rows=40
loops=1)
        Index Cond: (customer_id = 2718)
    -> Index Scan using idx_order_items_order_id on order_items oi (actual time=0.003..0.003
rows=3 loops=40)
        Index Cond: (order_id = o.order_id)
Planning Time: 0.445 ms
Execution Time: 0.274 ms
(11 rows)
```

The median fell from 225.37 ms to 0.30 ms. The join method changed from a hash join to a **nested loop**: for each of the 40 orders (`loops=40`), look up its lines in the new index, about 3 lines each (40×3 is close to the 108 lines found; `rows=3` is a rounded average). Looking up one order's lines, which a screen in an order-entry system does thousands of times a day, went from 79.62 ms to 0.06 ms.

The rule: in PostgreSQL, index every foreign key column that you join on or delete through. It's the single most common missing index.

2. Don't wrap the indexed column in a function

"How many orders came in December 2025?" There's an index on `order_date` . Two ways to write the filter:

```
-- version A: extract parts of the date
SELECT COUNT(*) FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 2025 AND EXTRACT(MONTH FROM order_date) = 12;

-- version B: a date range
SELECT COUNT(*) FROM orders
WHERE order_date >= DATE '2025-12-01' AND order_date < DATE '2026-01-01';
```

Both return 25,078. Their plans are very different:

QUERY PLAN

```
-----
Aggregate (actual time=109.158..109.159 rows=1 loops=1)
  -> Seq Scan on orders (actual time=101.616..108.010 rows=25078 loops=1)
        Filter: ((EXTRACT(year FROM order_date) = '2025'::numeric) AND (EXTRACT(month FROM
order_date) = '12'::numeric))
        Rows Removed by Filter: 689207
Planning Time: 0.279 ms
Execution Time: 109.196 ms
(6 rows)
```

QUERY PLAN

```
-----
Aggregate (actual time=2.840..2.841 rows=1 loops=1)
  -> Index Only Scan using idx_orders_order_date on orders (actual time=0.016..1.625 rows=25078
loops=1)
        Index Cond: ((order_date >= '2025-12-01'::date) AND (order_date < '2026-01-01'::date))
        Heap Fetches: 0
Planning Time: 0.303 ms
Execution Time: 2.876 ms
(6 rows)
```

Version A scans the whole table (median 100.95 ms). Version B uses the index (median 2.88 ms), 35 times faster. The index stores dates sorted, so it can find "from 1 December up to (but not including) 1 January" directly. It does not store "the month part of each date", so it can't help with `EXTRACT(MONTH ...)` = 12. A condition the database can answer from an index is called **sargable** (from *search argument*).

The same trap appears in many disguises. Each pair below returns the same rows, but only the second form can use a plain index on the column:

Not sargable	Sargable
<code>WHERE EXTRACT(YEAR FROM order_date) = 2025</code>	<code>WHERE order_date >= '2025-01-01' AND order_date < '2026-01-01'</code>
<code>WHERE order_date::text LIKE '2025-12%'</code>	the date range above
<code>WHERE UPPER(city) = 'PUNE'</code>	store city in one agreed case, then <code>WHERE city = 'Pune'</code>
<code>WHERE quantity * unit_price > 50000</code>	often unavoidable; see <i>expression indexes</i> below
<code>WHERE customer_name LIKE '%Hardware'</code>	<code>LIKE 'Sharma%'</code> can use an index; a leading <code>%</code> can't

The half-open range (`>=` the start, `<` the next start) is also the correct form for timestamps, which `BETWEEN ... AND '2025-12-31'` gets wrong by missing everything after midnight on the 31st.

If a function in the filter is truly needed, PostgreSQL and MySQL (8.0.13 and later) can index the expression itself: `CREATE INDEX idx_customers_upper_city ON customers (UPPER(city));`. The query must then use exactly the same expression.

3. Composite indexes: order matters

"Customer 4534's orders since 1 January 2025." This filters on two columns. A **composite index** stores several columns sorted together, like a phone book sorted by surname and then first name:

```
CREATE INDEX idx_orders_customer_date ON orders (customer_id, order_date);
```

With only the single-column index on `customer_id`, the database finds customer 2718's 40 orders and then checks each one's date (Filter: ... Rows Removed by Filter: 28). With the composite index, both conditions go into the `Index Cond`, and only the 12 matching rows are fetched. For a customer with 40 orders that's a small saving. For customer 4534, with 9,301 orders in 2025 alone, it's the difference between reading all their orders and reading only the ones you need.

The column order decides which questions an index can answer. An index on `(customer_id, order_date)` is sorted by customer first:

- ✓ `WHERE customer_id = 4534` (the first column alone).
- ✓ `WHERE customer_id = 4534 AND order_date >= '2025-01-01'` (equality on the first, range on the second).
- ✓ `WHERE customer_id = 4534 ORDER BY order_date DESC LIMIT 5` (the rows are already in date order within the customer).
- ✗ `WHERE order_date >= '2025-12-01'` on its own. The dates are sorted *within each customer*, not overall, like trying to find everyone named Priya in a phone book sorted by surname.

The usual guideline: put columns you test with `=` first, and a column you test with a range (`>=`, `<`, `BETWEEN`) last.

Watch out: extra indexes are often redundant. Once `(customer_id, order_date)` exists, a separate index on `customer_id` alone is almost never needed, because the composite index answers the same lookups. Dropping the redundant one saves space and speeds up writes. In the test run, for customer 2718's query, the planner ignored an index on `(order_date, customer_id)` and preferred the single `customer_id` index: the column-order rule in action.

4. Covering indexes: never touch the table

If every column a query needs is inside the index, the database can skip the table altogether. PostgreSQL lets you carry extra columns in an index without sorting by them, using `INCLUDE`:

```
CREATE INDEX idx_orders_customer_date_incl
  ON orders (customer_id, order_date) INCLUDE (status);
VACUUM ANALYZE orders;

EXPLAIN (ANALYZE, COSTS OFF)
SELECT order_date, status
FROM orders
WHERE customer_id = 4534 AND order_date >= DATE '2025-01-01';
```

QUERY PLAN

```
-----
Index Only Scan using idx_orders_customer_date_incl on orders (actual time=0.015..0.950 rows=9301 loops=1)
  Index Cond: ((customer_id = 4534) AND (order_date >= '2025-01-01'::date))
  Heap Fetches: 0
Planning Time: 0.270 ms
Execution Time: 1.282 ms
(5 rows)
```

Index Only Scan with **Heap Fetches: 0**: all 9,301 rows came from the index. The median fell from 3.69 ms (composite index, reading 2,047 table pages) to 1.24 ms. The **VACUUM** matters: PostgreSQL can only skip a table page if it knows every row on it is visible to everyone, and **VACUUM** records that. On a busy table, autovacuum does this in the background.

A covering index is a strong tool for one important, frequent query. It's a poor default, because every included column makes the index bigger.

5. Partial indexes: index only the rows you ask about

The warehouse screen asks one question all day: "Which orders are still pending, oldest first?" Only 3,930 of 714,285 orders are pending. An index on **status** would mostly store the word *Delivered*. A **partial index** stores only the rows that match its **WHERE** :

```
CREATE INDEX idx_orders_pending ON orders (order_date) WHERE status = 'Pending';

EXPLAIN (ANALYZE, COSTS OFF)
SELECT order_id, customer_id, order_date
FROM orders
WHERE status = 'Pending'
ORDER BY order_date;
```

QUERY PLAN

```
-----
Index Scan using idx_orders_pending on orders (actual time=0.009..0.410 rows=3930 loops=1)
Planning Time: 0.274 ms
Execution Time: 0.546 ms
(3 rows)
```

The median fell from 32.26 ms (scan everything, then sort) to 0.56 ms, and the plan has no sort step because the index already stores the pending orders in date order. The index is 48 kB, against 5,056 kB for an index on all order dates. As orders move from Pending to Shipped, they drop out of the index automatically. MySQL has no partial indexes (section 28.13).

6. Measure what indexes cost on writes

The same 500,000 order lines were inserted into two empty copies of **order_items** : one with no indexes, one with four. Three rounds each, median reported (**ch28_perf.py** has the code):

Table	Median time to insert 500,000 rows
no indexes	356 ms

Table	Median time to insert 500,000 rows
four indexes	2,122 ms

Six times slower. Each index is a sorted structure that must be updated for every row. That's why data loads often **drop indexes, load, and recreate them**, and why "add an index for every column someone might filter on" is bad advice for a table that receives thousands of writes a minute.

The experiments in one table

Question (riverstone_perf)	Before	After	Change made
One customer's orders (40 rows)	30.11 ms	0.08 ms	index on <code>orders.customer_id</code>
One customer's revenue (join)	225.37 ms	0.30 ms	index on <code>order_items.order_id</code>
One order's lines	79.62 ms	0.06 ms	same index
December's order count	100.95 ms	2.88 ms	rewrote <code>EXTRACT</code> as a date range
Big customer's 2025 orders	3.69 ms	1.24 ms	covering index with <code>INCLUDE (status)</code>
Pending orders, oldest first	32.26 ms	0.56 ms	partial index
Insert 500,000 lines	356 ms	2,122 ms	four indexes added (the cost)

A checklist for a slow query

1. **Confirm it's the query.** Time it in DBeaver or `psql (\timing on)`. A slow dashboard can be slow for reasons outside the database.
2. **Run `EXPLAIN (ANALYZE, BUFFERS)`** on a copy of production-sized data. `BUFFERS` adds how many pages each step read.
3. **Find the expensive step:** where the time jumps, where `Rows Removed by Filter` is huge, where estimated and actual rows differ by ten times or more.
4. **Estimates far off?** Run `ANALYZE table_name` so the statistics are current, then plan again.
5. **Scan with a selective filter?** Check the filter is sargable; then consider an index (composite if several columns).
6. **Big join without an index on the join column?** Index the foreign key.
7. **Still slow because the question itself reads millions of rows?** No index will fix that. Pre-aggregate (section 28.11).
8. **Measure again**, and write down the before and after. Then check that writes to the table haven't become a problem.

Real-life example: the index that took down checkout. A common story: an analyst notices a slow report and adds an index on a busy production table in the middle of the day. In PostgreSQL, a plain `CREATE INDEX` blocks inserts and updates to the table until it finishes, so on a very large table, orders stop being saved for minutes. `CREATE INDEX CONCURRENTLY` builds the index without blocking writes (slower, and it can't run inside a transaction). Changes to production tables go through a reviewed migration (section 28.12), not a query window.

28.7 Normalization, worked properly

Chapter 12 (section 12.1) gave you the intuition: **store each fact once, and link to it**. Normalization turns that intuition into rules you can check, called **normal forms**. Each form removes one kind of problem. You'll take one messy table all the way through the first three, in the lab database.

The starting point: an order sheet

Before Riverstone's order entry was cleaned up, someone kept a tracking sheet of January's orders like this. It's a table in the lab, built from real orders in the one-year database:

```
DROP DATABASE IF EXISTS riverstone_lab;
CREATE DATABASE riverstone_lab;
```

Connect to `riverstone_lab` (in DBeaver, a new connection; in `psql`, `\c riverstone_lab`), then:

```
CREATE TABLE order_sheet (
  order_id    INTEGER,
  order_date  DATE,
  customer    VARCHAR(100),
  city        VARCHAR(50),
  sales_rep   VARCHAR(100),
  products    VARCHAR(300)
);

INSERT INTO order_sheet VALUES
(10001, '2025-01-02', 'Patel Kitchenware', 'Ahmedabad', 'Neha Kulkarni', '108 Stackable Bin x10 @290'),
(10002, '2025-01-05', 'Green Leaf Hotels', 'Pune',      'Farah Khan',    '107 Lunch Box Set x55 @380'),
(10005, '2025-01-20', 'Green Leaf Hotels', 'Pune',      'Farah Khan',    '101 Storage Box 10L x15 @430 less 5%'),
(10006, '2025-01-21', 'Coastal Foods',    'Chennai',     'Rahul Mehta',   '102 Storage Box 25L x25 @750 less 5%'),
(10007, '2025-01-24', 'Sharma Hardware',  'Mumbai',     'Neha Kulkarni', '101 Storage Box 10L x45 @430; 102 Storage Box 25L x20 @750; 108 Stackable Bin x20 @290'),
(10008, '2025-01-26', 'Sharma Hardware',  'Mumbai',     'Neha Kulkarni', '101 Storage Box 10L x30 @430; 102 Storage Box 25L x25 @750; 107 Lunch Box Set x25 @380');
```

It's readable, and for a person it works. For a database it's a dead end. You can count the orders that mention a product:

```
SELECT COUNT(*) AS storage_box_10l_orders
FROM order_sheet
WHERE products LIKE '%Storage Box 10L%';
```

```
storage_box_10l_orders
-----
3
(1 row)
```

But you can't add up how many Storage Boxes were ordered, or the value of the orders, without taking the text apart. And the `LIKE` is fragile: an order typed as "Storage box 10 L" would be missed.

First normal form (1NF): one value per cell

A table is in **first normal form** when:

1. every cell holds a **single value** (no lists, no "x45 @430" packed into text), and
2. every row is **unique**, identified by a key.

The `products` column breaks rule 1: it holds a list of order lines, each with four facts (product, quantity, price, discount). The fix is one row per order line, with each fact in its own column:

```
CREATE TABLE order_lines_1nf (  
  order_id      INTEGER,  
  order_date    DATE,  
  customer      VARCHAR(100),  
  city          VARCHAR(50),  
  sales_rep     VARCHAR(100),  
  product_id    INTEGER,  
  product_name  VARCHAR(100),  
  quantity      INTEGER,  
  unit_price    NUMERIC(10,2),  
  discount_pct  NUMERIC(5,2),  
  PRIMARY KEY (order_id, product_id)  
);  
  
INSERT INTO order_lines_1nf VALUES  
(10001, '2025-01-02', 'Patel Kitchenware', 'Ahmedabad', 'Neha Kulkarni', 108, 'Stackable Bin',  
10, 290, 0),  
(10002, '2025-01-05', 'Green Leaf Hotels', 'Pune', 'Farah Khan', 107, 'Lunch Box Set',  
55, 380, 0),  
(10005, '2025-01-20', 'Green Leaf Hotels', 'Pune', 'Farah Khan', 101, 'Storage Box 10L',  
15, 430, 5),  
(10006, '2025-01-21', 'Coastal Foods', 'Chennai', 'Rahul Mehta', 102, 'Storage Box 25L',  
25, 750, 5),  
(10007, '2025-01-24', 'Sharma Hardware', 'Mumbai', 'Neha Kulkarni', 101, 'Storage Box 10L',  
45, 430, 0),  
(10007, '2025-01-24', 'Sharma Hardware', 'Mumbai', 'Neha Kulkarni', 102, 'Storage Box 25L',  
20, 750, 0),  
(10007, '2025-01-24', 'Sharma Hardware', 'Mumbai', 'Neha Kulkarni', 108, 'Stackable Bin',  
20, 290, 0),  
(10008, '2025-01-26', 'Sharma Hardware', 'Mumbai', 'Neha Kulkarni', 101, 'Storage Box 10L',  
30, 430, 0),  
(10008, '2025-01-26', 'Sharma Hardware', 'Mumbai', 'Neha Kulkarni', 102, 'Storage Box 25L',  
25, 750, 0),  
(10008, '2025-01-26', 'Sharma Hardware', 'Mumbai', 'Neha Kulkarni', 107, 'Lunch Box Set',  
25, 380, 0);
```

The key is `(order_id, product_id)`: one order can have many products, and a product appears once per order. Now arithmetic works:

```
SELECT COUNT(DISTINCT order_id) AS storage_box_10l_orders,  
       SUM(quantity)           AS units  
FROM order_lines_1nf  
WHERE product_id = 101;
```

```
storage_box_10l_orders | units  
-----+-----  
3 | 90  
(1 row)
```

(15 + 45 + 30 = 90 units. ✓) Most tables you meet in databases are already in 1NF. You'll meet ONF data mainly in spreadsheets, form exports, and JSON, and Chapter 14 shows how to split it.

The anomalies 1NF still allows

Look at what `order_lines_1nf` repeats: Sharma Hardware's city is stored on six rows, and "Storage Box 10L" is spelled out three times. Section 12.1 named the three problems this causes. Here's the first one, happening:

```
UPDATE order_lines_1nf
SET city = 'Thane'
WHERE order_id = 10008 AND product_id = 101;

SELECT customer, city, COUNT(*) AS lines
FROM order_lines_1nf
WHERE customer = 'Sharma Hardware'
GROUP BY customer, city;
```

customer	city	lines
Sharma Hardware	Mumbai	5
Sharma Hardware	Thane	1

(2 rows)

An **update anomaly**: someone corrected the city on one row, and now the same customer is in two cities. Nothing in the table's design prevents it. The other two are the **insert anomaly** (you can't record a new product until someone orders it, because the product has nowhere to live without an order line) and the **delete anomaly** (delete Patel Kitchenware's only order, and you lose the fact that they're in Ahmedabad).

Second normal form (2NF): every column depends on the whole key

To find the cause, ask of each column: **what does it depend on?** A column **depends on** another (a **functional dependency**, written $A \rightarrow B$) when knowing A tells you B.

- `order_date`, `customer`, `city`, `sales_rep` depend on `order_id` alone. Order 10007 has one date, whichever product you look at.
- `product_name` depends on `product_id` alone.
- `quantity`, `unit_price`, `discount_pct` depend on **both**: they describe one product within one order.

A table is in **second normal form** when it's in 1NF and **no column depends on only part of a composite key**. `order_lines_1nf` has **partial dependencies** (on `order_id` alone, and on `product_id` alone), so it isn't. The fix is to give each group of facts a table keyed by what it depends on:

```

UPDATE order_lines_1nf SET city = 'Mumbai' WHERE order_id = 10008;

CREATE TABLE orders_2nf AS
SELECT DISTINCT order_id, order_date, customer, city, sales_rep
FROM order_lines_1nf;

CREATE TABLE products_2nf AS
SELECT DISTINCT product_id, product_name
FROM order_lines_1nf;

CREATE TABLE order_lines_2nf AS
SELECT order_id, product_id, quantity, unit_price, discount_pct
FROM order_lines_1nf;

SELECT 'orders' AS table_name, COUNT(*) AS row_count FROM orders_2nf
UNION ALL SELECT 'products', COUNT(*) FROM products_2nf
UNION ALL SELECT 'order_lines', COUNT(*) FROM order_lines_2nf;

```

table_name	row_count
orders	6
products	4
order_lines	10

(3 rows)

(The first line undoes the bad update, so the split starts from clean data. `CREATE TABLE ... AS SELECT` copies the data but not keys; in a real design you'd add `PRIMARY KEY` and `REFERENCES` constraints, as Chapter 12 did.)

Six orders, four products, ten order lines. A product's name is now stored once, and a product can exist before anyone orders it.

Watch out: `unit_price` is not a partial dependency here. It's tempting to move `unit_price` to the products table, because every product has a price. But the price *charged on an order* is a fact about that order line: next year's list price will change, and last year's orders must keep what the customer actually paid. The list price belongs in `products`; the charged price belongs in `order_lines`. Riverstone's real tables do exactly this. Normalization is about what the data *means*, which the column names alone won't tell you.

Third normal form (3NF): no column depends on another non-key column

`orders_2nf` still stores `city` on every order. City depends on `order_id`, but only *through* the customer: `order_id` → customer → `city`. That's a **transitive dependency**. A table is in **third normal form** when it's in 2NF and **no non-key column depends on another non-key column**. A common way to remember 2NF and 3NF together: every non-key column must depend on *the key, the whole key, and nothing but the key*.

Move the customer's facts into their own table, with a **surrogate key** (an id with no business meaning, created only to identify the row), and keep only the id on the order:

```

CREATE TABLE customers_3nf (
    customer_id INTEGER PRIMARY KEY,
    customer    VARCHAR(100) NOT NULL UNIQUE,
    city        VARCHAR(50)
);

INSERT INTO customers_3nf
SELECT ROW_NUMBER() OVER (ORDER BY customer), customer, city
FROM (SELECT DISTINCT customer, city FROM orders_2nf) AS c;

CREATE TABLE orders_3nf AS
SELECT o.order_id, o.order_date, c.customer_id, o.sales_rep
FROM orders_2nf AS o
JOIN customers_3nf AS c ON c.customer = o.customer;

SELECT * FROM customers_3nf ORDER BY customer_id;

```

customer_id	customer	city
1	Coastal Foods	Chennai
2	Green Leaf Hotels	Pune
3	Patel Kitchenware	Ahmedabad
4	Sharma Hardware	Mumbai

(4 rows)

Now Sharma Hardware's city is stored once. The update anomaly from earlier can't happen, because there's only one row to update. (`sales_rep` has the same problem as `city` , since a rep's details depend on the rep, and exercise 9 asks you to fix it.)

Proving nothing was lost

Normalization splits tables without losing information: joining them back must reproduce the original. Reconcile the revenue:

```

SELECT 'flat 1NF table' AS source,
    ROUND(SUM(quantity * unit_price * (1 - discount_pct / 100)), 2) AS net_revenue
FROM order_lines_1nf
UNION ALL
SELECT '3NF tables joined',
    ROUND(SUM(l.quantity * l.unit_price * (1 - l.discount_pct / 100)), 2)
FROM order_lines_2nf AS l
JOIN orders_3nf      AS o ON o.order_id = l.order_id
JOIN customers_3nf   AS c ON c.customer_id = o.customer_id
JOIN products_2nf    AS p ON p.product_id = l.product_id;

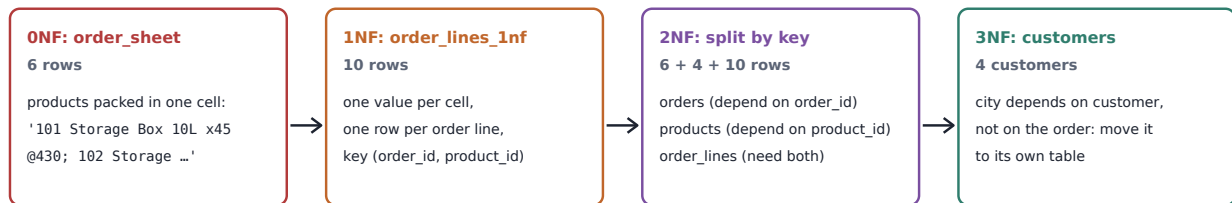
```

source	net_revenue
flat 1NF table	129040.00
3NF tables joined	129040.00

(2 rows)

Both ₹129,040. ✓ Check one line by hand: order 10005 is $15 \times ₹430$ less 5% = ₹6,127.50.

Normalization, one rule at a time (the same six orders)



Problems removed: 0NF → 1NF you can filter and sum. 1NF → 2NF a product's name is stored once.
2NF → 3NF a customer's city is stored once, so it can't disagree with itself. Every step: the net revenue still totals ₹129,040.

Figure 28.5 – Each normal form removes one kind of repetition. The total never changes.

Beyond 3NF, and when to stop

There are stricter forms. **Boyce–Codd normal form (BCNF)** tightens 3NF for tables with several overlapping candidate keys, and fourth and fifth normal forms deal with multi-valued facts. They're worth knowing by name for interviews; in business databases, a design in 3NF is almost always in BCNF too.

3NF is the standard target for **transactional systems** (the ERP, the CRM), where data is written constantly and must stay consistent. Reporting systems deliberately bend the rules, for speed and simplicity. That's sections 28.9 and 28.11.

Normal form	The rule	Problem it removes	Riverstone fix
1NF	one value per cell; rows unique	can't filter, count, or add	one row per order line
2NF	no column depends on part of the key	product names repeated; can't add a product without an order	products table
3NF	no column depends on a non-key column	customer's city repeated and able to disagree	customers table

28.8 The grain discipline

Chapter 12 (section 12.10) named the idea: the **grain** of a table is what one row represents. Chapter 12 used it to explain fan-out, and promised a whole discipline. Here it is, in four habits.

Habit 1: state the grain in one sentence, before anything else

Every table, view, and query result should have a grain you can say out loud:

Table	Grain	Key that proves it
orders	one order	order_id
order_items	one product line within one order	order_item_id, and (order_id, product_id) should also be unique
sales_lines (view)	one non-cancelled order line	order_id, product_id

Table	Grain	Key that proves it
sales_targets	one calendar month	target_month
lead_stage_history	one lead entering one stage	(lead_id, stage)
staff	one person	staff_id

If you can't write the sentence, you don't understand the table yet, and any total you compute from it is a guess.

Habit 2: test the grain, don't assume it

A grain is a claim about the data, and data breaks claims. The test is a `GROUP BY` on the columns that should be unique:

```
SELECT order_id, product_id, COUNT(*) AS copies
FROM order_items
GROUP BY order_id, product_id
HAVING COUNT(*) > 1;
```

```
order_id | product_id | copies
-----+-----+-----
(0 rows)
```

No rows: each product appears at most once per order, so the grain holds. Chapter 13's Pattern 3 found the opposite in `leads`, where 43 rows held 30 real leads. Chapter 47 turns tests like this into automated checks that run every day, and Chapter 32 writes them as dbt tests.

Habit 3: never join tables of different grains and then add

Here's the trap in its most common business form. "Show January's order lines next to January's target." The target is monthly; the lines are daily. Join them and sum the target:

```
SELECT ROUND(SUM(t.target_revenue)) AS target_summed_after_join
FROM sales_lines AS sl
JOIN sales_targets AS t
  ON t.target_month = DATE_TRUNC('month', sl.order_date)
WHERE sl.order_date < DATE '2025-02-01';
```

```
target_summed_after_join
-----
4500000
(1 row)
```

January's target is ₹300,000, not ₹4,500,000. The join copied the one monthly target onto each of January's 15 order lines, and `SUM` added all 15 copies ($15 \times ₹300,000$). The query ran without error and returned a believable-looking number. This is the fan-out trap from section 12.10 in a new form, and it's the reason for habit 4.

Habit 4: bring both sides to the same grain, then join

Aggregate the finer table up to the coarser grain first, then join one row to one row:

```
WITH monthly_sales AS (  
    SELECT DATE_TRUNC('month', order_date)::date AS month,  
           SUM(net_revenue) AS revenue  
    FROM sales_lines  
    GROUP BY 1  
)  
SELECT m.month,  
       ROUND(m.revenue) AS revenue,  
       ROUND(t.target_revenue) AS target,  
       ROUND(100 * m.revenue / t.target_revenue, 1) AS pct_of_target  
FROM monthly_sales AS m  
JOIN sales_targets AS t ON t.target_month = m.month  
WHERE m.month < DATE '2025-04-01'  
ORDER BY m.month;
```

month	revenue	target	pct_of_target
2025-01-01	202640	300000	67.5
2025-02-01	253664	300000	84.6
2025-03-01	278008	320000	86.9

(3 rows)

January: ₹202,640 of ₹300,000 is 67.5%. ✓ Both sides now have the grain "one month", so the join can't multiply anything.

Interview extra point. When an interviewer gives you two tables and a question, say the grain of each table before you write a join: "orders is one row per order, order_items is one row per line, so if I join them I'll aggregate the lines first." It shows the habit that prevents the most common wrong answer, and it's the kind of move Part VIII's SQL bank (Chapter 71) scores as an extra point.

28.9 Dimensional modeling: facts, dimensions, and the star schema

Normalized tables are built for *writing*: each fact once, so updates are safe. Analysts mostly *read*, and reading a normalized database means long chains of joins, repeated in every report and often subtly wrong. **Dimensional modeling**, popularized by Ralph Kimball, organizes the same data for reading. It's how most data warehouses, Power BI models (Chapter 16), and dbt marts (Chapter 32) are laid out.

Facts and dimensions

Every business question has two kinds of words in it. "What was **net revenue** by **customer segment** and **quarter**?"

- **Facts** (or **measures**) are the numbers you add up: net revenue, quantity, cost. They come from business events: an order line, a payment, a delivery.

- **Dimensions** are the words you slice by: segment, quarter, product category, sales rep. They describe the *who*, *what*, *where*, and *when* of each event.

A **fact table** holds one row per event at a declared grain, with the measures and a key for each dimension. A **dimension table** holds one row per member of the dimension (one per customer, one per day), with every descriptive column a report might need. Arrange the dimensions around the fact table and draw lines to it, and the diagram looks like a star: a **star schema**.

The four design steps

Kimball's method is four questions, asked in order:

1. **Choose the business process.** Not "sales data" but a specific event: *taking orders*.
2. **Declare the grain.** *One row per non-cancelled order line*. This is section 28.8's habit, and it comes before choosing any columns.
3. **Identify the dimensions.** What describes one order line? When (date), who bought (customer), what (product), who sold (sales rep).
4. **Identify the facts.** What numbers belong to one order line? Quantity, net revenue, product cost.

The answers give Riverstone's first star schema. You'll build it in a schema called `dw` (for *data warehouse*) inside `riverstone_2025`, next to the source tables, as section 12.13's schemas lesson suggested.

Riverstone's 2025 sales as a star schema (schema `dw`)

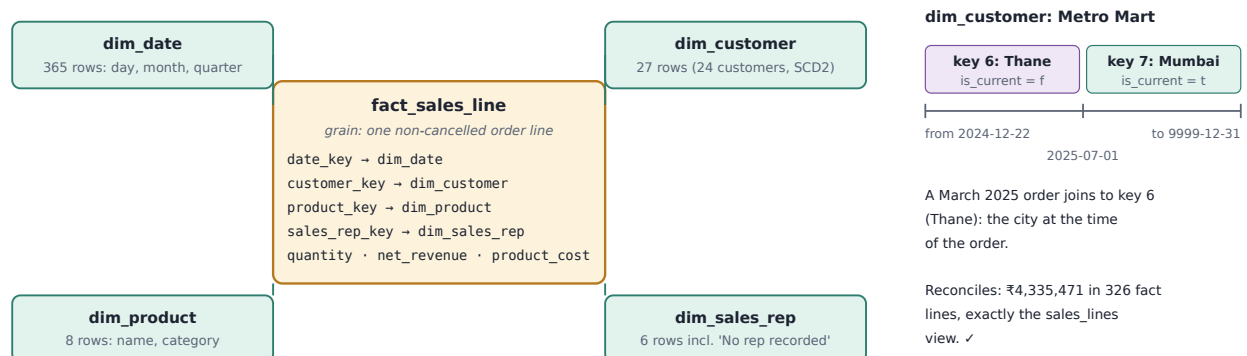


Figure 28.6 – The fact table holds the events; each dimension describes one side of them. Metro Mart has two customer rows because it moved.

The date dimension

The most useful dimension, and the one people most often skip, is a **date dimension**: one row per calendar day, with every way the business describes a date already worked out.

```

DROP SCHEMA IF EXISTS dw CASCADE;
CREATE SCHEMA dw;

CREATE TABLE dw.dim_date (
    date_key          INTEGER PRIMARY KEY,
    full_date         DATE NOT NULL UNIQUE,
    day_name          VARCHAR(9) NOT NULL,
    month_start       DATE NOT NULL,
    month_name        VARCHAR(9) NOT NULL,
    quarter_label     CHAR(7) NOT NULL,
    is_weekend        BOOLEAN NOT NULL
);

INSERT INTO dw.dim_date
SELECT TO_CHAR(d, 'YYYYMMDD')::INTEGER,
       d::date,
       TRIM(TO_CHAR(d, 'Day')),
       DATE_TRUNC('month', d)::date,
       TRIM(TO_CHAR(d, 'Month')),
       TO_CHAR(d, 'YYYY-"Q"Q'),
       EXTRACT(ISODOW FROM d) IN (6, 7)
FROM generate_series(DATE '2025-01-01', DATE '2025-12-31', INTERVAL '1 day') AS d;

```

How it works:

- `generate_series` (Chapter 13) produces every day of 2025, including days with no orders, so a report on "orders by weekday" or "days without sales" has every day to count.
- `date_key` is a whole number like `20250315`. It's readable, sorts correctly, and joins fast. Using a meaningful number for dates is a common exception to the rule that surrogate keys carry no meaning.
- Each descriptive column is computed once here instead of in every report: `TO_CHAR(d, 'Day')` gives the weekday name, `'YYYY-"Q"Q'` gives `2025-Q1`. Real date dimensions add fiscal year (April to March, for most Indian companies), holiday flags, and week numbers. When finance decides that the festive season starts in October, you change one table, not fifty reports.

Product and sales rep dimensions

```

CREATE TABLE dw.dim_product (
    product_key      SERIAL PRIMARY KEY,
    product_id       INTEGER NOT NULL UNIQUE,
    product_name     VARCHAR(100) NOT NULL,
    category         VARCHAR(30) NOT NULL
);
INSERT INTO dw.dim_product (product_id, product_name, category)
SELECT product_id, product_name, category FROM products ORDER BY product_id;

CREATE TABLE dw.dim_sales_rep (
    sales_rep_key    SERIAL PRIMARY KEY,
    employee_id      INTEGER UNIQUE,
    rep_name         VARCHAR(100) NOT NULL,
    job_title        VARCHAR(50) NOT NULL
);
INSERT INTO dw.dim_sales_rep (employee_id, rep_name, job_title)
VALUES (NULL, 'No rep recorded', 'Unknown');
INSERT INTO dw.dim_sales_rep (employee_id, rep_name, job_title)
SELECT employee_id, employee_name, job_title FROM employees ORDER BY employee_id;

```

Each dimension gets its own **surrogate key** (`product_key` , `sales_rep_key`), separate from the source system's id. Section 28.10 shows why that separation is essential for customers. The sales rep dimension also has a row for "No rep recorded", because 11 orders have no rep (10 of them not cancelled); the fact table can then always point to a real dimension row, and reports show an honest "No rep recorded" instead of silently dropping those orders in an inner join.

The customer dimension

Customers are harder, because they change: in 2025 Patel Kitchenware was reclassified from Wholesale to Retail, Metro Mart moved from Thane to Mumbai, and Harbour Traders moved from Retail to Wholesale. The ERP's `customers` table only holds the values as they are now; the history is in `customer_changes` :

```
SELECT change_id, customer_id, changed_on, column_name, old_value, new_value
FROM customer_changes
ORDER BY changed_on;
```

change_id	customer_id	changed_on	column_name	old_value	new_value
1	2	2025-04-01	segment	Wholesale	Retail
2	5	2025-07-01	city	Thane	Mumbai
3	11	2025-09-01	segment	Retail	Wholesale

(3 rows)

The dimension below keeps a separate row for each version of each customer. Section 28.10 explains the design (it's a *type 2 slowly changing dimension*); for now, read the build as "one row per customer per period in which their details stayed the same".

```

CREATE TABLE dw.dim_customer (
    customer_key    SERIAL PRIMARY KEY,
    customer_id     INTEGER NOT NULL,
    customer_name   VARCHAR(100) NOT NULL,
    city            VARCHAR(50),
    segment         VARCHAR(30) NOT NULL,
    valid_from      DATE NOT NULL,
    valid_to        DATE NOT NULL,
    is_current      BOOLEAN NOT NULL
);

INSERT INTO dw.dim_customer
    (customer_id, customer_name, city, segment, valid_from, valid_to, is_current)
WITH version_starts AS (
    SELECT customer_id, signup_date AS valid_from FROM customers
    UNION
    SELECT customer_id, changed_on FROM customer_changes
),
versions AS (
    SELECT customer_id,
           valid_from,
           LEAD(valid_from, 1, DATE '9999-12-31')
             OVER (PARTITION BY customer_id ORDER BY valid_from) AS valid_to
    FROM version_starts
)
SELECT v.customer_id,
       c.customer_name,
       COALESCE(
           (SELECT ch.new_value FROM customer_changes AS ch
            WHERE ch.customer_id = v.customer_id AND ch.column_name = 'city'
              AND ch.changed_on <= v.valid_from
            ORDER BY ch.changed_on DESC LIMIT 1),
           (SELECT ch.old_value FROM customer_changes AS ch
            WHERE ch.customer_id = v.customer_id AND ch.column_name = 'city'
            ORDER BY ch.changed_on LIMIT 1),
           c.city)
       AS city,
       COALESCE(
           (SELECT ch.new_value FROM customer_changes AS ch
            WHERE ch.customer_id = v.customer_id AND ch.column_name = 'segment'
              AND ch.changed_on <= v.valid_from
            ORDER BY ch.changed_on DESC LIMIT 1),
           (SELECT ch.old_value FROM customer_changes AS ch
            WHERE ch.customer_id = v.customer_id AND ch.column_name = 'segment'
            ORDER BY ch.changed_on LIMIT 1),
           c.segment)
       AS segment,
       v.valid_from,
       v.valid_to,
       v.valid_to = DATE '9999-12-31'
       AS is_current
FROM versions AS v
JOIN customers AS c ON c.customer_id = v.customer_id
ORDER BY v.customer_id, v.valid_from;

```

How it works:

- `version_starts` lists the dates on which each customer's details could begin: their signup date, plus every change date.
- `versions` uses `LEAD` (section 13.5) to end each version the day the next one starts. The last version gets the far-future end date `9999-12-31`, meaning "still current".
- For each version, the subqueries find the city (and segment) in effect: the new value of the latest change on or before the version's start; if there wasn't one yet, the *old* value of the first change;

and if there were never any changes, the current value from `customers` .

The fact table

```
CREATE TABLE dw.fact_sales_line (  
  order_item_id INTEGER PRIMARY KEY,  
  order_id      INTEGER NOT NULL,  
  date_key      INTEGER NOT NULL REFERENCES dw.dim_date (date_key),  
  customer_key  INTEGER NOT NULL REFERENCES dw.dim_customer (customer_key),  
  product_key   INTEGER NOT NULL REFERENCES dw.dim_product (product_key),  
  sales_rep_key INTEGER NOT NULL REFERENCES dw.dim_sales_rep (sales_rep_key),  
  quantity      INTEGER NOT NULL,  
  net_revenue   NUMERIC(12,2) NOT NULL,  
  product_cost  NUMERIC(12,2) NOT NULL  
);  
  
INSERT INTO dw.fact_sales_line  
SELECT oi.order_item_id,  
       o.order_id,  
       TO_CHAR(o.order_date, 'YYYYMMDD')::INTEGER,  
       dc.customer_key,  
       dp.product_key,  
       dr.sales_rep_key,  
       oi.quantity,  
       oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100),  
       oi.quantity * p.unit_cost  
FROM orders AS o  
JOIN order_items AS oi ON oi.order_id = o.order_id  
JOIN products   AS p  ON p.product_id = oi.product_id  
JOIN dw.dim_customer AS dc  
  ON dc.customer_id = o.customer_id  
  AND o.order_date >= dc.valid_from AND o.order_date < dc.valid_to  
JOIN dw.dim_product AS dp ON dp.product_id = oi.product_id  
JOIN dw.dim_sales_rep AS dr  
  ON dr.employee_id IS NOT DISTINCT FROM o.sales_rep_id  
WHERE o.status <> 'Cancelled';
```

The critical line is the join to `dw.dim_customer`: `o.order_date >= dc.valid_from AND o.order_date < dc.valid_to` . Each order line gets the key of the customer version **in effect on the order date**. Once that key is stored, the fact row never needs to look at the date range again. `IS NOT DISTINCT FROM` is a null-safe equals: unlike `=` , it treats two NULLs as equal, so orders with no rep match the "No rep recorded" row.

Check the build

A warehouse you haven't reconciled is a warehouse nobody should trust. Three checks:

```
SELECT 'dim_date' AS table_name, COUNT(*) AS row_count FROM dw.dim_date  
UNION ALL SELECT 'dim_customer', COUNT(*) FROM dw.dim_customer  
UNION ALL SELECT 'dim_product', COUNT(*) FROM dw.dim_product  
UNION ALL SELECT 'dim_sales_rep', COUNT(*) FROM dw.dim_sales_rep  
UNION ALL SELECT 'fact_sales_line', COUNT(*) FROM dw.fact_sales_line;
```

table_name	row_count
dim_date	365
dim_customer	27
dim_product	8
dim_sales_rep	6
fact_sales_line	326

(5 rows)

365 days. 27 customer rows: 24 customers, plus one extra version for each of the three who changed. ✓

```
SELECT customer_key, customer_name, city, segment, valid_from, valid_to, is_current
FROM dw.dim_customer
WHERE customer_id IN (2, 5, 11)
ORDER BY customer_id, valid_from;
```

customer_key	customer_name	city	segment	valid_from	valid_to	is_current
2	Patel Kitchenware	Ahmedabad	Wholesale	2024-12-15	2025-04-01	f
3	Patel Kitchenware	Ahmedabad	Retail	2025-04-01	9999-12-31	t
6	Metro Mart	Thane	Retail	2024-12-22	2025-07-01	f
7	Metro Mart	Mumbai	Retail	2025-07-01	9999-12-31	t
13	Harbour Traders	Kochi	Retail	2025-04-09	2025-09-01	f
14	Harbour Traders	Kochi	Wholesale	2025-09-01	9999-12-31	t

(6 rows)

Each changed customer has one closed version and one current version, and each version ends on the day the next begins. ✓

```
SELECT (SELECT ROUND(SUM(net_revenue)) FROM sales_lines) AS view_total,
       (SELECT ROUND(SUM(net_revenue)) FROM dw.fact_sales_line) AS fact_total,
       (SELECT COUNT(*) FROM sales_lines) AS view_lines,
       (SELECT COUNT(*) FROM dw.fact_sales_line) AS fact_lines;
```

view_total	fact_total	view_lines	fact_lines
4335471	4335471	326	326

(1 row)

₹4,335,471 and 326 lines in both. ✓ If any order line had failed to find a customer version (a gap between one version's end and the next one's start, for instance), the inner join would have dropped it and the fact total would be lower. This one query catches that.

Querying the star

Every report now has the same simple shape: the fact table, joined to the dimensions you need, grouped by their columns.

```

SELECT d.quarter_label,
       c.segment,
       ROUND(SUM(f.net_revenue)) AS net_revenue
FROM dw.fact_sales_line AS f
JOIN dw.dim_date      AS d ON d.date_key = f.date_key
JOIN dw.dim_customer AS c ON c.customer_key = f.customer_key
GROUP BY d.quarter_label, c.segment
ORDER BY d.quarter_label, c.segment;

```

quarter_label	segment	net_revenue
2025-Q1	Hospitality	149040
2025-Q1	Retail	356290
2025-Q1	Wholesale	228982
2025-Q2	Hospitality	252170
2025-Q2	Retail	271031
2025-Q2	Wholesale	203367
2025-Q3	Hospitality	307278
2025-Q3	Retail	384294
2025-Q3	Wholesale	428718
2025-Q4	Hospitality	435551
2025-Q4	Retail	562000
2025-Q4	Wholesale	756751

(12 rows)

No `DATE_TRUNC`, no `CASE`, no joins through orders to reach customers. The quarters add up to the year: Q4 alone is ₹1,754,302 (435,551 + 562,000 + 756,751), 40.5% of the total, which matches Chapter 13's festive-season finding.

Star or snowflake?

In a star schema, each dimension is one wide table, even if that repeats values: `category` is stored on every product row. A **snowflake schema** normalizes the dimensions too (`dim_product` points to a `dim_category` table), so the diagram branches like a snowflake.

	Star	Snowflake
Dimension tables	one wide table per dimension	split into sub-dimensions
Joins in a report	one per dimension	more
Repetition in dimensions	yes, deliberately	less
Easier for BI tools and business users	yes	no
Typical use	most warehouses and Power BI models	very large dimensions with many shared levels

Dimension tables are small (Riverstone's biggest has 27 rows; a large company's customer dimension might have a few million), so the repetition costs little, and the simpler joins help everyone who writes reports. **Default to a star.** Power BI's documentation recommends star schemas for its models, and Chapter 16 builds on this one.

Other kinds of fact tables

`fact_sales_line` is a **transaction fact table**: one row per event, added once and never changed. Two other kinds cover questions it can't answer well:

- A **periodic snapshot** records the state of something at regular intervals: stock on hand at each warehouse at the end of each day, or receivables outstanding at each month-end. You can't add snapshots across time (adding daily stock levels makes no sense), but you can average them or take the latest.
- An **accumulating snapshot** has one row per process instance, updated as it moves through fixed milestones: one row per order with the dates it was placed, invoiced, shipped, delivered, and paid. It's the natural home for Chapter 3's order-to-cash durations.

And one more design term you'll hear: **conformed dimensions** are dimensions shared by several fact tables with the same keys and meanings. If the sales facts and the payments facts both use the same `dim_customer` and `dim_date`, a report can put revenue and collections side by side and trust that "Retail, 2025-Q3" means the same thing in both.

28.10 Slowly changing dimensions

Customers move, get reclassified, change their names. Products move between categories. Sales reps change teams. Dimensions change slowly (compared with facts, which arrive every minute), and how you handle those changes decides whether last year's reports still say what they said last year. The standard catalog of techniques is called **slowly changing dimension (SCD) types**. Three matter in practice.

To see each one clearly, go back to the lab and use the example from section 12.1: suppose Sharma Hardware moves from Mumbai to Thane, effective 1 April 2026. A nightly extract from the ERP (`customer_extract`) brings the new city, along with a brand-new customer, Coastal Foods.

```

CREATE TABLE dim_customer_t1 (
    customer_id INTEGER PRIMARY KEY,
    customer    VARCHAR(100) NOT NULL,
    city        VARCHAR(50),
    segment     VARCHAR(30)
);

INSERT INTO dim_customer_t1 VALUES
(1, 'Sharma Hardware', 'Mumbai', 'Retail'),
(3, 'Green Leaf Hotels', 'Pune', 'Hospitality');

CREATE TABLE customer_extract (
    customer_id INTEGER PRIMARY KEY,
    customer    VARCHAR(100) NOT NULL,
    city        VARCHAR(50),
    segment     VARCHAR(30)
);

INSERT INTO customer_extract VALUES
(1, 'Sharma Hardware', 'Thane', 'Retail'),
(3, 'Green Leaf Hotels', 'Pune', 'Hospitality'),
(4, 'Coastal Foods', 'Chennai', 'Wholesale');

```

Type 1: overwrite

Type 1 replaces the old value with the new one. No history is kept. It's right for **corrections** (a misspelled name, a wrong PIN code) and for attributes nobody reports history on.

The tidiest way to apply an extract is **MERGE** (PostgreSQL 15 and later), which updates rows that match and inserts rows that don't, in one statement:

```

MERGE INTO dim_customer_t1 AS d
USING customer_extract AS x
ON d.customer_id = x.customer_id
WHEN MATCHED AND (d.city IS DISTINCT FROM x.city OR d.segment IS DISTINCT FROM x.segment) THEN
    UPDATE SET city = x.city, segment = x.segment
WHEN NOT MATCHED THEN
    INSERT (customer_id, customer, city, segment)
    VALUES (x.customer_id, x.customer, x.city, x.segment);

SELECT * FROM dim_customer_t1 ORDER BY customer_id;

```

customer_id	customer	city	segment
1	Sharma Hardware	Thane	Retail
3	Green Leaf Hotels	Pune	Hospitality
4	Coastal Foods	Chennai	Wholesale

(3 rows)

How it works: `USING` names the incoming data and `ON` says how to match it. `WHEN MATCHED AND ...` updates only rows that really changed (`IS DISTINCT FROM` treats two NULLs as equal, so unchanged NULLs don't count as changes). `WHEN NOT MATCHED` inserts new customers. Sharma Hardware is now in Thane, and **every past sale of Sharma Hardware now reports as Thane**, including sales made in Mumbai in 2025. For a correction, that's exactly right. For a real move, it rewrites history.

Type 2: add a new row

Type 2 keeps history by closing the current row and inserting a new one. Each row is a *version*, with its own surrogate key and a validity period. This is what `dw.dim_customer` did in section 28.9.

```
CREATE TABLE dim_customer_t2 (  
    customer_key SERIAL PRIMARY KEY,  
    customer_id  INTEGER NOT NULL,  
    customer     VARCHAR(100) NOT NULL,  
    city         VARCHAR(50),  
    segment      VARCHAR(30),  
    valid_from   DATE NOT NULL,  
    valid_to     DATE NOT NULL DEFAULT '9999-12-31',  
    is_current    BOOLEAN NOT NULL DEFAULT TRUE  
);  
  
INSERT INTO dim_customer_t2 (customer_id, customer, city, segment, valid_from) VALUES  
(1, 'Sharma Hardware', 'Mumbai', 'Retail', '2024-11-04'),  
(3, 'Green Leaf Hotels', 'Pune', 'Hospitality', '2024-09-08');  
  
CREATE UNIQUE INDEX one_current_row ON dim_customer_t2 (customer_id) WHERE is_current;
```

The columns that make it type 2:

- **customer_key**: the surrogate key. `customer_id` (the **natural key** or **business key** from the ERP) is no longer unique, because one customer can have several versions. Facts store `customer_key`, which points to exactly one version.
- **valid_from and valid_to**: the period the version was true. A half-open period (from `valid_from`, up to but not including `valid_to`) means consecutive versions never overlap and never leave a gap.
- **is_current**: a convenience flag for "the latest version".
- The **partial unique index** (section 28.6) makes the database enforce the rule that each customer has at most one current row.

Applying the extract is two steps, in one transaction so nobody ever sees a customer with no current row:

```

BEGIN;

-- step 1: close the current version of customers whose details changed
UPDATE dim_customer_t2 AS d
SET valid_to = DATE '2026-03-31',
    is_current = FALSE
FROM customer_extract AS x
WHERE d.customer_id = x.customer_id
    AND d.is_current
    AND (d.city IS DISTINCT FROM x.city OR d.segment IS DISTINCT FROM x.segment);

-- step 2: insert a new current version for every customer without one
INSERT INTO dim_customer_t2 (customer_id, customer, city, segment, valid_from)
SELECT x.customer_id, x.customer, x.city, x.segment, DATE '2026-04-01'
FROM customer_extract AS x
LEFT JOIN dim_customer_t2 AS d
    ON d.customer_id = x.customer_id AND d.is_current
WHERE d.customer_id IS NULL;

COMMIT;

SELECT customer_key, customer_id, customer, city, valid_from, valid_to, is_current
FROM dim_customer_t2
ORDER BY customer_id, valid_from;

```

customer_key	customer_id	customer	city	valid_from	valid_to	is_current
1	1	Sharma Hardware	Mumbai	2024-11-04	2026-03-31	f
3	1	Sharma Hardware	Thane	2026-04-01	9999-12-31	t
2	3	Green Leaf Hotels	Pune	2024-09-08	9999-12-31	t
4	4	Coastal Foods	Chennai	2026-04-01	9999-12-31	t

(4 rows)

How it works: step 1 closes Sharma Hardware's Mumbai version. Green Leaf Hotels didn't change, so it's left alone. Step 2 then looks for extract rows with no current version: that's Sharma Hardware (whose current version step 1 closed) and Coastal Foods (brand new). Both get a new row. Green Leaf still has its current row, so the `LEFT JOIN ... IS NULL` skips it.

Simplification note. This version compares two columns and hard-codes the dates. A production load compares every tracked column (often by comparing a hash of them), takes the dates from the extract, handles customers deleted from the source, and runs as a scheduled job. Chapter 32 shows dbt's *snapshots*, which implement type 2 this way from a short configuration.

Joining facts to a type 2 dimension

Three orders: one before the move, two after.

```

CREATE TABLE fact_sales (
    order_id      INTEGER,
    order_date    DATE,
    customer_id   INTEGER,
    net_revenue   NUMERIC(12,2)
);

INSERT INTO fact_sales VALUES
(20001, '2026-03-18', 1, 42000),
(20002, '2026-04-09', 1, 35500),
(20003, '2026-04-10', 3, 18000);

SELECT f.order_id,
       f.order_date,
       d.customer_key,
       d.city AS city_at_order_time,
       f.net_revenue
FROM fact_sales AS f
JOIN dim_customer_t2 AS d
  ON d.customer_id = f.customer_id
 AND f.order_date >= d.valid_from
 AND f.order_date < d.valid_to
ORDER BY f.order_id;

```

order_id	order_date	customer_key	city_at_order_time	net_revenue
20001	2026-03-18	1	Mumbai	42000.00
20002	2026-04-09	3	Thane	35500.00
20003	2026-04-10	2	Pune	18000.00

(3 rows)

The March order belongs to Mumbai; the April order to Thane. This is a **point-in-time join**. In a warehouse you do it once, when loading facts, and store the `customer_key`, as `dw.fact_sales_line` did.

Now the trap. Join on the business key alone, forgetting the dates:

```

SELECT d.city, SUM(f.net_revenue) AS net_revenue
FROM fact_sales AS f
JOIN dim_customer_t2 AS d ON d.customer_id = f.customer_id
GROUP BY d.city
ORDER BY d.city;

```

city	net_revenue
Mumbai	77500.00
Pune	18000.00
Thane	77500.00

(3 rows)

Total revenue should be ₹95,500. This shows ₹173,000. Each of Sharma Hardware's two orders matched both versions, so ₹77,500 (₹42,000 + ₹35,500) was counted in Mumbai *and* in Thane. It's the fan-out trap again: a type 2 dimension's grain is *one customer version*, not *one customer*, and joining on `customer_id` alone ignores that.

Type 3: add a "previous" column

Type 3 keeps limited history in extra columns: the current value and the one before it.

```
CREATE TABLE dim_customer_t3 (  
  customer_id      INTEGER PRIMARY KEY,  
  customer         VARCHAR(100) NOT NULL,  
  city            VARCHAR(50),  
  previous_city    VARCHAR(50),  
  city_changed_on  DATE  
);  
  
INSERT INTO dim_customer_t3 VALUES (1, 'Sharma Hardware', 'Mumbai', NULL, NULL);  
  
UPDATE dim_customer_t3  
SET previous_city = city,  
    city         = 'Thane',  
    city_changed_on = DATE '2026-04-01'  
WHERE customer_id = 1;  
  
SELECT * FROM dim_customer_t3;
```

customer_id	customer	city	previous_city	city_changed_on
1	Sharma Hardware	Thane	Mumbai	2026-04-01

(1 row)

In PostgreSQL's `UPDATE`, every right-hand side sees the row's *old* values, so `previous_city = city` copies Mumbai even though `city` is assigned in the same statement.

Type 3 answers one specific question well: "Show this year's sales under both the old and the new territory map." It can't tell you where a customer was two moves ago, and it doesn't split sales by date.

Choosing a type

Type	What happens on a change	History kept	Use it for	Watch out for
1	overwrite	none	corrections; attributes nobody reports history on (phone, contact name)	silently restates past reports
2	close old row, add new row	full	attributes reports are sliced by, where history matters (segment, city, sales territory, product category)	join facts through the surrogate key or with a date range, never on the business key alone
3	move old value to a "previous" column	one step back	a planned, one-time reorganization compared side by side	only one previous value

A single dimension usually mixes types: in Riverstone's customer dimension, a corrected phone number would be type 1, and a segment change type 2.

Why type 2 changes the numbers: a real Riverstone result

Back in `riverstone_2025`: "What did each segment sell in 2025?" There are two honest answers, and the star schema can give both. **As it was** uses the version on each fact row. **As it is** re-labels every sale with each customer's current segment.

```

WITH as_it_was AS (
    SELECT c.segment, SUM(f.net_revenue) AS revenue
    FROM dw.fact_sales_line AS f
    JOIN dw.dim_customer AS c ON c.customer_key = f.customer_key
    GROUP BY c.segment
),
as_it_is AS (
    SELECT cur.segment, SUM(f.net_revenue) AS revenue
    FROM dw.fact_sales_line AS f
    JOIN dw.dim_customer AS c ON c.customer_key = f.customer_key
    JOIN dw.dim_customer AS cur ON cur.customer_id = c.customer_id AND cur.is_current
    GROUP BY cur.segment
)
SELECT w.segment,
    ROUND(w.revenue) AS revenue_as_it_was,
    ROUND(i.revenue) AS revenue_as_it_is,
    ROUND(i.revenue - w.revenue) AS difference
FROM as_it_was AS w
JOIN as_it_is AS i ON i.segment = w.segment
ORDER BY w.segment;

```

segment	revenue_as_it_was	revenue_as_it_is	difference
Hospitality	1144039	1144039	0
Retail	1573615	1488774	-84842
Wholesale	1617817	1702659	84842

(3 rows)

Reading it. Retail's 2025 revenue is ₹1,573,615 as it was, or ₹1,488,774 if you re-label history with today's segments: ₹84,842 apart. Reconcile the difference by hand from the monthly figures: Patel Kitchenware's orders before 1 April (₹2,900 in January and ₹35,138 in March, ₹38,038) move into Retail, and Harbour Traders' orders from April to August (₹122,880) move out. ₹38,038 – ₹122,880 = – ₹84,842. ✓ Both columns total ₹4,335,471.

Neither column is wrong. They answer different questions: "How did the Retail team's customers perform at the time?" versus "How much have the customers who are Retail today bought?" A type 1 dimension can only give the second answer, and it would give it without telling anyone. **What to tell Anita:** say which view a report uses, in its title.

28.11 Denormalization trade-offs

Denormalization means deliberately storing data more than once, or storing results that could be calculated, to make reading faster or simpler. A star schema is already denormalized (category repeated on every product row). This section is about the other common form: storing **pre-computed results**.

The problem: a question that reads everything

"Monthly revenue by product category, for the last three years." On `riverstone_perf` that joins 700,070 non-cancelled orders to 1,888,466 lines and groups them into 144 rows. No index helps, because the question needs every row. Median time: 2,343 ms. For one analyst once a month, fine. For a dashboard

opened by forty people every morning, it's forty queries of two and a half seconds, each reading 140 MB, for numbers that only change once a day.

A summary table or materialized view

A **materialized view** is a view whose result is stored as a table and refreshed on request:

```
CREATE MATERIALIZED VIEW monthly_category_revenue AS
SELECT DATE_TRUNC('month', o.order_date)::date AS month,
       p.category,
       SUM(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100)) AS net_revenue
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
JOIN products AS p ON p.product_id = oi.product_id
WHERE o.status <> 'Cancelled'
GROUP BY 1, 2;

REFRESH MATERIALIZED VIEW monthly_category_revenue;
```

Once built, it's read like any table:

```
SELECT month, category, ROUND(net_revenue) AS net_revenue
FROM monthly_category_revenue
WHERE month = DATE '2025-12-01'
ORDER BY net_revenue DESC;
```

month	category	net_revenue
2025-12-01	Storage	292916949
2025-12-01	Industrial	286297690
2025-12-01	Furniture	230339308
2025-12-01	Kitchen	226362946

(4 rows)

(The amounts are far larger than Riverstone's real 2025 figures in `riverstone_2025`: the large database was generated to have realistic *volume*, not realistic totals.)

The measured trade-off on the test machine:

	Time
Live query (median of 5)	2,343 ms
Build the materialized view	1,906 ms
Read a year from the materialized view (median of 7)	0.04 ms
REFRESH MATERIALIZED VIEW	1,926 ms

One refresh a night costs about two seconds; every read after that is effectively free. The price is **staleness**: until the next refresh, the view doesn't show today's orders. For a monthly trend that's fine. For "orders taken in the last hour", it isn't.

The trade-offs, laid out

Technique	Faster reads because	You pay with	Good for
Star schema (repeated attributes)	fewer joins	more storage; updates to a dimension attribute touch many rows	all reporting
Summary table / materialized view	results already computed	staleness; refresh time; another object to maintain	dashboards, recurring reports on large data
Copying a column onto another table (for example customer segment onto orders)	saves a join	the two copies can disagree (section 28.7's update anomaly)	rarely; only with an automatic process keeping it in sync
Storing derived columns (for example <code>net_revenue</code> on order lines)	no calculation at read time	must be recomputed if inputs change	expensive, stable calculations

Three rules keep denormalization safe:

1. **Normalize first.** The source of truth stays normalized. Denormalized copies are derived from it and can always be rebuilt.
2. **Denormalize for a measured problem**, not in advance. The table above started with a 2.3-second query, not a guess.
3. **Automate the refresh and make staleness visible.** Put "data as of 06:00" on the dashboard. A summary nobody refreshes becomes a wrong number that everyone trusts.

Dialect note. PostgreSQL's `REFRESH MATERIALIZED VIEW CONCURRENTLY` lets people keep reading during a refresh (it needs a unique index on the view). MySQL has no materialized views: create a summary table and refresh it with a scheduled `DELETE ... INSERT ... SELECT` in a transaction, or an event (section 28.13). Warehouses such as Snowflake and BigQuery have their own materialized views with automatic refresh (Chapter 49), and dbt builds summary tables as *models* on a schedule (Chapter 32).

28.12 Schema migrations: changing a live database safely

Section 12.13 ended with a checklist for changing a table's structure, and promised the tools teams use to do it every week without breaking things. Here they are.

The problem migrations solve

Riverstone now has the ERP database, a reporting copy, a test copy, and every developer's laptop copy. A developer adds a column on their laptop. Does the test copy have it? Production? Which changes were run on production, in which order, by whom? When the answer lives in people's memories and old emails, databases **drift** apart, and one Monday a report fails in production that worked perfectly in test.

A **migration** is a small script that moves a database's structure from one version to the next. The discipline is:

1. **Every change is a file**, never typed into a query window on a shared database.
2. **Files are numbered**, and always applied in number order.
3. **The database records which migrations it has run**, in a history table.
4. **A migration never changes once applied**. A mistake is fixed by a *new* migration.
5. **Every migration is reviewed** (a pull request, Chapter 26) and run on test before production.

Building the idea by hand

Before using a tool, see that there's no magic. Continue in the lab:

```
CREATE TABLE schema_migrations (  
  version      INTEGER PRIMARY KEY,  
  description  VARCHAR(100) NOT NULL,  
  checksum     CHAR(32) NOT NULL,  
  applied_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

Migrations 1 and 2 create a table and add its first rows. Each runs in a transaction together with its history row, so either both happen or neither does. (In a real project, `checksum` is a hash of the migration file's contents; here it's computed from a stand-in string.)

```
BEGIN;  
CREATE TABLE suppliers (  
  supplier_id  INTEGER PRIMARY KEY,  
  supplier_name VARCHAR(100) NOT NULL,  
  city         VARCHAR(50) NOT NULL  
);  
INSERT INTO schema_migrations (version, description, checksum)  
VALUES (1, 'create suppliers', md5('V1__create_suppliers.sql contents'));  
COMMIT;  
  
BEGIN;  
INSERT INTO suppliers VALUES  
(1, 'Western Polymers', 'Vapi'),  
(2, 'Deccan Cartons', 'Pune'),  
(3, 'Sagar Labels', 'Mumbai');  
INSERT INTO schema_migrations (version, description, checksum)  
VALUES (2, 'seed suppliers', md5('V2__seed_suppliers.sql contents'));  
COMMIT;
```

Migration 3 adds a column. Section 12.13's rule applies: a new `NOT NULL` column on a table with rows needs a `DEFAULT`, or existing rows break the constraint:

```
BEGIN;  
ALTER TABLE suppliers ADD COLUMN payment_terms_days INTEGER NOT NULL DEFAULT 30;  
INSERT INTO schema_migrations (version, description, checksum)  
VALUES (3, 'add payment terms', md5('V3__add_payment_terms.sql contents'));  
COMMIT;  
  
SELECT version, description FROM schema_migrations ORDER BY version;
```

```

version | description
-----+-----
      1 | create suppliers
      2 | seed suppliers
      3 | add payment terms
(3 rows)

```

Any copy of the database can now answer "which version are you on?" with `SELECT MAX(version) FROM schema_migrations;`, and a tool can apply only the files with a higher number.

And here's the failure the `DEFAULT` rule prevents, in a migration someone forgot to test against real data:

```

BEGIN;
ALTER TABLE suppliers ADD COLUMN gst_number VARCHAR(15) NOT NULL;

```

```

ERROR: column "gst_number" of relation "suppliers" contains null values

```

Because the migration runs inside a transaction, the error leaves nothing half-done: no column, no history row. PostgreSQL can roll back structure changes like this; MySQL can't (section 28.13), which is one more reason to test migrations on a copy first.

Dialect note: the same migration in MySQL. MySQL 8.0 doesn't raise this error. It adds the column and fills the existing rows with the type's implicit default, an empty string for `VARCHAR`, even in strict mode (tested on 8.0.46). The migration "succeeds", and three suppliers now have a GST number of `' '` that no validation will ever flag. Same script, same intent, a silent data problem in one database and a loud error in the other: test migrations on every database engine you deploy to.

Renaming without breaking anyone: expand, then contract

Suppose the team wants to rename `suppliers.city` to `supplier_city`. `ALTER TABLE ... RENAME COLUMN` takes a moment, but every report, application, and dbt model that reads `city` breaks at that moment. On a system other people depend on, split the change into steps that are each safe on their own. This is the **expand and contract** pattern (also called *parallel change*):

1. **Expand:** add the new column and copy the data. Old readers still work.
2. **Migrate readers:** change each report and application to use the new column, in their own time. (Keep writers filling both columns meanwhile.)
3. **Contract:** once nothing reads the old column, drop it.

```

BEGIN;
ALTER TABLE suppliers ADD COLUMN supplier_city VARCHAR(50);
UPDATE suppliers SET supplier_city = city;
INSERT INTO schema_migrations (version, description, checksum)
VALUES (4, 'expand: add supplier_city', md5('V4 contents'));
COMMIT;

SELECT supplier_name, city, supplier_city FROM suppliers ORDER BY supplier_id;

```

supplier_name	city	supplier_city
Western Polymers	Vapi	Vapi
Deccan Cartons	Pune	Pune
Sagar Labels	Mumbai	Mumbai

(3 rows)

Days or weeks later, after every reader has moved:

```
BEGIN;
ALTER TABLE suppliers ALTER COLUMN supplier_city SET NOT NULL;
ALTER TABLE suppliers DROP COLUMN city;
INSERT INTO schema_migrations (version, description, checksum)
VALUES (5, 'contract: drop old city column', md5('V5 contents'));
COMMIT;

SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'suppliers'
ORDER BY ordinal_position;
```

column_name	data_type	is_nullable
supplier_id	integer	NO
supplier_name	character varying	NO
payment_terms_days	integer	NO
supplier_city	character varying	NO

(4 rows)

Migration tools

Nobody writes those `INSERT INTO schema_migrations` lines by hand in a real project. A **migration tool** reads a folder of numbered files, compares it with the history table, and applies what's missing. The ideas above are all the tools do; they differ in style:

Tool	How migrations are written	Common with
Flyway (Redgate)	plain SQL files named <code>V1__create_suppliers.sql</code> , <code>V2__seed_suppliers.sql</code>	Java and SQL-first teams; widely used with PostgreSQL, MySQL, SQL Server, Oracle
Liquibase	<i>changelogs</i> in SQL, XML, YAML, or JSON, with <i>changesets</i>	enterprises needing detailed change tracking across many databases
Alembic	Python files, usually generated from SQLAlchemy models	Python applications (Chapter 29)
Framework tools (Django, Rails, Laravel, Entity Framework)	code in the application's language	teams whose database belongs to one application
dbt	not a migration tool: it rebuilds analytical models from <code>SELECT</code> statements	warehouses (Chapter 32)

Flyway shows the conventions most clearly. Versioned migration files are named with a `V`, a version number, two underscores, and a description, like `V3__add_payment_terms.sql`, and are applied once each, in version order. Flyway records each one in a **schema history table** (`flyway_schema_history`)

with a **checksum** of the file, so if someone edits a file after it was applied, `flyway validate` reports the mismatch instead of silently letting environments drift. `flyway info` lists applied and pending migrations. **Repeatable migrations**, named `R__description.sql`, have no version and are re-applied whenever their checksum changes, which suits views. (Versions current at the time of writing are listed in the chapter's *Tools* section; check Redgate's documentation for the edition you use.)

A migration project for Riverstone's reporting database looks like this:

```
reporting-db/
├── migrations/
│   ├── V1__create_suppliers.sql
│   ├── V2__seed_suppliers.sql
│   ├── V3__add_payment_terms.sql
│   ├── V4__expand_add_supplier_city.sql
│   ├── V5__contract_drop_city.sql
│   └── R__view_supplier_summary.sql
└── README.md
```

The companion file `migrate.py` is a 50-line runner that applies a folder like this to PostgreSQL in the same way, with checksums, so you can watch the mechanism work before adopting a real tool. (It applies only `V` files; the `R__` file shows Flyway's naming.) Here it is on an empty database, run twice, and then run again after someone edited `V3` by adding a comment line:

```
# terminal, in companion/ch28 (create the empty database riverstone_lab_migrations first)
$ python3 migrate.py --db riverstone_lab_migrations migrations
applied V1__create_suppliers.sql
applied V2__seed_suppliers.sql
applied V3__add_payment_terms.sql
applied V4__expand_add_supplier_city.sql
applied V5__contract_drop_city.sql
database is at version 5
$ python3 migrate.py --db riverstone_lab_migrations migrations
nothing to apply; database is at version 5
$ python3 migrate.py --db riverstone_lab_migrations migrations
checksum mismatch: V3__add_payment_terms.sql was changed after it was applied. Write a new
migration instead.
```

The second run is the point of the history table: running migrations is **idempotent** (running it again changes nothing). The third run is the point of the checksum: a quiet edit to an applied migration would mean this database and every other copy now disagree about what "version 3" means, so the runner refuses to continue.

Reviewing a migration

Before approving a migration, a reviewer asks:

1. **Does it run on a copy with realistic data?** Constraints that pass on an empty test table fail on real rows, as `gst_number` did.
2. **Does it lock a busy table?** Adding an index without `CONCURRENTLY`, changing a column's type, or adding a `NOT NULL` column can block writes on large tables. Test the duration on production-sized data.
3. **Does it break a reader?** Renames and drops go through expand and contract. Search the reports, views, and dbt models for the column name.

4. **Is data preserved?** A type change (`VARCHAR` to `INTEGER`) or a split column needs a data check before and after: row counts and a total.
5. **Is there a way back?** Either a written reverse migration, or a backup taken immediately before, with the restore tested (section 2.9).
6. **Is it one change?** Small migrations are quick to review and quick to fix. "V7__misc_changes.sql" is a warning sign.

28.13 The same SQL in MySQL

Most of this chapter runs in MySQL 8.0 unchanged: recursive CTEs, `ROWS` and `RANGE` frames (including intervals), `FIRST_VALUE` and `LAST_VALUE`, `NTILE`, named windows, indexes, `EXPLAIN`, normalization, star schemas, and type 2 dimensions built with `UPDATE` and `INSERT`. The companion file `ch28_queries_mysql.sql` has a tested MySQL version of every query. Here's what changes.

Setting up

Load `riverstone_2025_setup_mysql.sql`, then `ch28_2025_addons_mysql.sql`. For the large database, run `generate_riverstone_perf.py` and load `perf_data/load_mysql.sql` with `mysql --local-infile=1` (the server setting `local_infile` must be `ON`).

What changes

Feature	PostgreSQL	MySQL 8.0
Recursive CTE	<code>WITH RECURSIVE</code>	same; recursion stops after 1,000 levels by default (<code>cte_max_recursion_depth</code>)
Column sizes in a recursive CTE	taken from both parts; types must match	taken from the anchor; <code>CAST</code> growing strings in the anchor
<code>SEARCH</code> / <code>CYCLE</code> clauses	yes (14+)	no; build a path string and check it
<code>GROUPS</code> frames, <code>EXCLUDE</code>	yes	no
<code>RANGE</code> with interval	<code>INTERVAL '6 days'</code>	<code>INTERVAL 6 DAY</code>
<code>PERCENTILE_CONT</code>	yes	no; use <code>NTILE</code> or <code>ROW_NUMBER</code> with <code>COUNT</code>
Foreign key indexes	not automatic	created automatically by InnoDB
Partial indexes	yes	no (a functional index on a <code>CASE</code> expression can imitate one)
Covering index	<code>INCLUDE (...)</code>	no <code>INCLUDE</code> ; add the columns to the index key
<code>EXPLAIN</code> output	text tree	<code>EXPLAIN FORMAT=TREE</code> ; <code>EXPLAIN ANALYZE</code> (8.0.18+)
<code>MERGE</code>	yes (15+)	no; <code>INSERT ... ON DUPLICATE KEY UPDATE</code> for type 1
Materialized views	yes	no; summary tables
Transactional structure changes	yes: <code>ALTER</code> can be rolled back	no: each <code>ALTER</code> / <code>CREATE</code> commits immediately

Feature	PostgreSQL	MySQL 8.0
Adding a NOT NULL column with no default to a table with rows	error	succeeds, filling rows with ' ', 0, or a zero date

A tree in MySQL

With no `SEARCH` clause, order an org chart by building a **sort path** of zero-padded ids. The anchor's `CAST(... AS CHAR(200))` sets the column's size; without it, MySQL sizes the column from the anchor's four-character first value, and the query stops with ERROR 1406 (22001): *Data too long for column 'sort_path' at row 1* as soon as the path grows.

```
WITH RECURSIVE org AS (
  SELECT staff_id, staff_name, 1 AS level,
         CAST(LPAD(staff_id, 4, '0') AS CHAR(200)) AS sort_path
  FROM staff
  WHERE manager_id IS NULL
  UNION ALL
  SELECT s.staff_id, s.staff_name, o.level + 1,
         CONCAT(o.sort_path, '/', LPAD(s.staff_id, 4, '0'))
  FROM staff AS s
  JOIN org AS o ON s.manager_id = o.staff_id
)
SELECT CONCAT(REPEAT('  ', level - 1), staff_name) AS org_chart
FROM org
WHERE sort_path LIKE '0100/0110%'
ORDER BY sort_path;
```

```
+-----+
| org_chart |
+-----+
|      Anita Rao      |
|      Vikram Singh   |
|      Neha Kulkarni  |
|      Rahul Mehta    |
|      Farah Khan     |
|      Meera Iyer     |
|      Arjun Nair     |
|      Divya Krishnan |
|      Vivek Chandran |
|      Sneha Pillai   |
|      Pooja Desai    |
|      Nisha Bhatt    |
|      Aditya Verma   |
|      Rohit Kamat    |
|      Sandeep Gill   |
|      Karan Ahuja    |
|      Ritu Bansal    |
+-----+
```

`WHERE sort_path LIKE '0100/0110%'` keeps Anita Rao's branch (staff 110 under 100). The same path string is also the loop check: in the recursive part, add `WHERE LOCATE(LPAD(s.staff_id, 4, '0'), o.sort_path) = 0`.

MySQL's safety limit stops runaway recursion for you:

```
WITH RECURSIVE numbers AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM numbers
)
SELECT COUNT(*) FROM numbers;
```

```
ERROR 3636 (HY000): Recursive query aborted after 1001 iterations. Try increasing
@@cte_max_recursion_depth to a larger value.
```

For a legitimate deep walk, raise it for your session: `SET SESSION cte_max_recursion_depth = 10000;`

Frames

`RANGE` with an interval works:

```
SELECT order_date,
       COUNT(*) OVER (ORDER BY order_date
                      RANGE BETWEEN INTERVAL 6 DAY PRECEDING AND CURRENT ROW) AS orders_last_7_days
FROM orders
WHERE order_date BETWEEN '2025-02-14' AND '2025-02-21'
ORDER BY order_date, order_id;
```

order_date	orders_last_7_days
2025-02-16	2
2025-02-16	2
2025-02-17	4
2025-02-17	4
2025-02-20	6
2025-02-20	6

`GROUPS` does not:

```
SELECT order_date,
       COUNT(*) OVER (ORDER BY order_date GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW) AS
orders_in_frame
FROM orders
LIMIT 3;
```

```
ERROR 1235 (42000): This version of MySQL doesn't yet support 'GROUPS'
```

Plans in MySQL

MySQL's tree format reads the same way as PostgreSQL's, inside out. Here are the December queries from section 28.6 on `riverstone_perf` in MySQL, with an index on `order_date` (recorded output, test machine):

```
EXPLAIN ANALYZE
SELECT COUNT(*) FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 2025 AND EXTRACT(MONTH FROM order_date) = 12;
```

```
-> Aggregate: count(0) (cost=143147 rows=1) (actual time=113..113 rows=1 loops=1)
    -> Filter: ((extract(year from orders.order_date) = 2025) and (extract(month from
orders.order_date) = 12)) (cost=71842 rows=713052) (actual time=107..112 rows=25078 loops=1)
        -> Covering index scan on orders using idx_orders_order_date (cost=71842 rows=713052)
            (actual time=0.519..69.8 rows=714285 loops=1)
```

```
EXPLAIN ANALYZE
SELECT COUNT(*) FROM orders
WHERE order_date >= '2025-12-01' AND order_date < '2026-01-01';
```

```
-> Aggregate: count(0) (cost=14488 rows=1) (actual time=6.74..6.74 rows=1 loops=1)
    -> Filter: ((orders.order_date >= DATE'2025-12-01') and (orders.order_date < DATE'2026-01-01'))
(cost=9672 rows=48152) (actual time=0.31..5.64 rows=25078 loops=1)
        -> Covering index range scan on orders using idx_orders_order_date over ('2025-12-01' <=
order_date < '2026-01-01') (cost=9672 rows=48152) (actual time=0.309..3.22 rows=25078 loops=1)
```

The lesson transfers exactly. The `EXTRACT` version reads all 714,285 index entries (MySQL is clever enough to scan the small index instead of the table, but it still reads everything), with a median of 115 ms; the range reads only December's 25,078, with a median of 6.38 ms. And MySQL needed no index for the single-customer lookup: its automatic foreign-key index on `customer_id` gave a median of 0.08 ms straight after loading.

Type 1 without MERGE

MySQL's upsert updates a row when the primary key already exists, and inserts it otherwise:

```
INSERT INTO dim_customer_t1 (customer_id, customer, city, segment)
SELECT customer_id, customer, city, segment
FROM customer_extract AS x
ON DUPLICATE KEY UPDATE city = x.city, segment = x.segment;
```

The type 2 load (close, then insert) works as in section 28.10, except that MySQL's multi-table `UPDATE` is written `UPDATE dim_customer_t2 AS d JOIN customer_extract AS x ON ... SET ...`. Both versions are in `ch28_queries_mysql.sql`.

Watch out: MySQL can't roll back an ALTER TABLE. In MySQL, `CREATE`, `ALTER`, and `DROP` commit immediately, even inside `BEGIN ... COMMIT`. A migration that fails halfway leaves the first half applied. Keep each MySQL migration to one structure change, and test it on a copy.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Recursive CTE with no loop protection	Query runs until cancelled, or MySQL stops at 1,000 levels	Add a depth limit, a path check, or PostgreSQL's <code>CYCLE</code> clause
Anchor and recursive part with different column types	<i>recursive query ... has type numeric(8,3) in non-recursive term</i> (PostgreSQL); <i>Data too long</i> (MySQL)	<code>CAST</code> the anchor's columns to the wider type

Mistake	Symptom	Fix
Adding BOM quantities instead of multiplying them down the tree	Material needs far too low for multi-level products	Multiply each child's quantity by its parent's running quantity
Walking a hierarchy with orphan rows	Some people or parts silently missing from the result	Compare the walk's row count with the table; <code>LEFT JOIN</code> to find missing managers
<code>LAST_VALUE</code> with the default frame	"Latest" value equals the current row	Frame <code>ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING</code>
<code>ROWS</code> frame on data with missing days	"Last 7 days" really covers several weeks	<code>RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW</code>
Tuning on a tiny test database	Every plan is a sequential scan and every query is instant; production is slow	Test plans on production-sized data
Function wrapped around an indexed column	Seq Scan with a huge Rows Removed by Filter, although an index exists	Rewrite as a range, or create an expression index
No index on a PostgreSQL foreign key	Joins and deletes on the child table slow down as it grows	Index foreign key columns you join or delete through
Composite index in the wrong column order	Index exists but isn't used for the query	Equality columns first, range column last
Indexing every column "just in case"	Loads and inserts get slower; indexes bigger than the table	Index for measured, frequent queries; drop redundant indexes
Trusting estimated costs as timings	Surprise when a "cheap" plan is slow	Use <code>EXPLAIN ANALYZE</code> ; costs are relative units
<code>EXPLAIN ANALYZE</code> on a <code>DELETE</code> or <code>UPDATE</code>	Rows really changed	Wrap in <code>BEGIN; ... ROLLBACK;</code>
Moving the charged price into the products table while normalizing	Past orders show today's prices	Separate list price (product) from price charged (order line)
Joining tables of different grains, then summing	Totals several times too large, with no error (₹4,500,000 target for January)	Aggregate to a common grain first
Joining facts to a type 2 dimension on the business key	Revenue counted once per customer version	Join on the surrogate key, or on the key plus the date range
Using type 1 for attributes reports are sliced by	Last year's segment or territory totals change without new data	Use type 2 and say which view the report shows
Materialized view nobody refreshes	Dashboard numbers stop moving; everyone trusts them	Schedule the refresh and show "data as of"
Editing a migration that was already applied	Databases drift; tools report checksum errors	Never edit; add a new migration
Renaming or dropping a column in one step	Reports and applications break the moment it runs	Expand, migrate readers, then contract
Adding <code>NOT NULL</code> without a default to a table with rows	PostgreSQL: <i>column ... contains null values</i> ; MySQL: no error, rows silently get <code>' '</code> or <code>0</code>	Add a <code>DEFAULT</code> , or add nullable, backfill, then set <code>NOT NULL</code>

In the real world: the dashboard that got slower every month

It's March 2026. Vikram Singh opens the sales dashboard in the Monday review and waits. The customer page, the December comparison, and the three-year trend each take a few seconds, and the whole team watches the spinner. *"This was instant when we launched it,"* he says. *"Can someone look at it before next Monday?"*

Meera Iyer takes it. She's tempted to start adding indexes, but she has seen that go wrong: an index on the busy orders table during working hours would block order entry while it builds. So she works the checklist from section 28.6, on a copy of the reporting database, which (like `riverstone_perf`) holds three years of orders.

First, find where the time goes. She copies the dashboard's four queries out of the report and runs each with `EXPLAIN (ANALYZE, BUFFERS)`. The numbers below are the chapter's measured medians, from queries of the same shape:

Dashboard tile	Median	What the plan shows
Customer page: one customer's revenue	225.37 ms	Seq Scan on order_items : 1.9 million lines read for 108
December orders compared with last year	100.95 ms	Seq Scan on orders with <code>EXTRACT(YEAR ...)</code> in the filter
Pending orders list	32.26 ms	Seq Scan , 710,355 rows removed
Three-year monthly trend by category	2,343 ms	Hash joins over every non-cancelled line

The trend tile is 86.7% of the 2,702 ms total. That's the headline for Vikram: one tile is the problem, and it's a different kind of problem from the other three.

Second, fix each for the right reason.

- *Customer page.* `order_items.order_id` has no index. The ERP vendor's MySQL version had one automatically; the PostgreSQL reporting copy doesn't. One index: 0.30 ms.
- *December comparison.* No index is missing: the filter can't use the one on `order_date`. She rewrites `EXTRACT(YEAR FROM order_date) = 2025 AND EXTRACT(MONTH FROM order_date) = 12` as a date range. 2.88 ms, no database change needed.
- *Pending list.* A partial index on pending orders: 48 kB, 0.56 ms.
- *Trend.* No index can help a question that reads three years of lines. The numbers only change once a day, so a materialized view refreshed at 06:00 turns it into a 144-row table. Refresh: about 2 seconds a night. Reading it: 0.04 ms.

Third, check she hasn't broken anything. The trend from the materialized view matches the live query for every month and category. She measures the cost to writes too: the reporting copy is loaded nightly, and the chapter's test showed four indexes making a large insert six times slower. Two new indexes and one partial index add seconds to a load that runs once a night, which is acceptable, and she writes that down.

Fourth, ship it properly. She doesn't run anything on the reporting database herself. She writes two migrations, `V14__index_order_items_order_id.sql` (with `CREATE INDEX CONCURRENTLY`, in its own non-transactional migration) and `V15__monthly_category_revenue.sql`, and a change to the dashboard's

December query. The data engineer reviews them against section 28.12's checklist and applies them on Wednesday evening. The refresh joins the nightly job, and the trend tile gets a caption: "Data as of 06:00 today."

What she tells Vikram: *"The dashboard's four tiles took about 2.7 seconds of database time; they now take under 4 milliseconds. Three fixes were missing or misused indexes. The fourth was the trend chart re-adding three years of orders every time anyone opened it; it now reads a summary refreshed every morning, so it won't show today's orders until tomorrow. Nothing changes for anyone entering orders."*

Notice what made it work. The measurement came first, so she fixed the tile that mattered most. Each fix matched its cause, rather than "add indexes until it's fast". Every change went through a reviewed migration. And the one trade-off, a trend that's a day old, was put in front of the person who owns the decision.

Tools

Versions used for this chapter, checked in September 2026:

- **PostgreSQL 16** (tested on 16 on Ubuntu 24.04). Everything here works in PostgreSQL 15 and later; `CYCLE` and `SEARCH` need 14 or later, `MERGE` needs 15 or later.
 - **MySQL 8.0** (tested on 8.0.46). `EXPLAIN ANALYZE` needs 8.0.18 or later, and expression indexes need 8.0.13 or later. The features used behave the same in the 8.4 long-term-support release.
 - **DBeaver** for running queries. Its *Explain execution plan* button shows plans as a tree; `EXPLAIN (ANALYZE, BUFFERS)` in the editor gives the full text.
 - **Python 3.12** for `generate_riverstone_perf.py` and `migrate.py`. Both use only the standard library; `migrate.py` calls the `psql` client.
 - **A plan visualizer (optional)**. Several free tools turn PostgreSQL's `EXPLAIN (ANALYZE, FORMAT JSON)` output into a diagram. Paste plans only from practice data, never from a production database containing customer data.
 - **Migration tools**. Flyway (Redgate; the free edition covers versioned SQL migrations), Liquibase, and Alembic for Python projects. Check the current release and edition details on each tool's documentation site before you install. Flyway was not run for this chapter, because its installer couldn't be downloaded in the test environment; its conventions are described from Redgate's documentation, and `migrate.py` demonstrates the same mechanism.
 - **Companion files for this chapter**: `ch28_2025_addons.sql` and `ch28_2025_addons_mysql.sql` (staff, parts, BOM, customer changes), `generate_riverstone_perf.py` (the large database), `ch28_perf.py` (reruns every timing experiment), `ch28_star_schema.sql` (the `dw` schema from section 28.9), `migrate.py` with its `migrations/` folder, and `ch28_queries_mysql.sql` (every query in MySQL).
-

The project: a star schema with history

Goal: a small, reconciled data warehouse for one business process, with a type 2 dimension, that a BI tool can read without knowing anything about the source tables.

Option A: your own work data. Choose a process you report on (orders, tickets, payments) and a dimension that changes over time (customer segment, account owner, region). Work on a copy, never the production database, and remove or mask personal data before you start. Keep the project on your own machine unless your employer has agreed otherwise.

Option B: Riverstone. Use `riverstone_2025` and the tables from this chapter.

Steps:

1. **Write the design on one page:** the business process, the grain of the fact table in one sentence, the dimensions, the facts, and for each dimension attribute whether changes are type 1 or type 2, with a reason.
2. **Build the date dimension** for your period, with at least day name, month, quarter, and fiscal year (April to March).
3. **Build the other dimensions**, each with a surrogate key. Add an "unknown" row wherever the source has missing values.
4. **Build the type 2 dimension** from your change history (Riverstone: `customer_changes`). Check that each business key has exactly one current row and no gaps or overlaps between versions.
5. **Load the fact table** with a point-in-time join to the type 2 dimension.
6. **Reconcile.** Row counts and totals against the source (Riverstone: ₹4,335,471 and 326 lines). Hand-check one fact row end to end.
7. **Answer three business questions** from the star: one by date attribute, one "as it was" versus "as it is" by the changing attribute, one that uses the unknown row.
8. **Make it repeatable.** Put the build into numbered migration files for the structures and one load script for the data, so the whole warehouse can be dropped and rebuilt from nothing.

Deliverables: the one-page design, the SQL files, the reconciliation query and its output, and a half-page note with the three answers.

Stretch goals:

- Add a second fact table at a different grain (monthly targets, or invoices) that shares `dim_date`, and build a report that puts both side by side correctly.
- Write the type 2 load as an *incremental* script: given tomorrow's extract, it closes and inserts only what changed. Run it twice to prove that the second run changes nothing.
- Load the star into Power BI (Chapter 16) and confirm that the model view shows a star.
- Build the same warehouse in MySQL and list every change you had to make.

You've got it when...

- ☐ I can write a recursive CTE that walks down or up a hierarchy, and explain the anchor, the recursive part, and when it stops.
 - ☐ I multiply quantities down a bill of materials and can reconcile a rolled-up cost by hand.
 - ☐ I protect recursive queries against loops, and I check that no rows went missing.
 - ☐ I choose `ROWS`, `RANGE`, or `GROUPS` deliberately, and use intervals for calendar windows.
 - ☐ I can read an `EXPLAIN ANALYZE` plan from the inside out and point to the expensive step.
 - ☐ I know why a function on an indexed column prevents index use, and how to rewrite it.
 - ☐ I can justify an index (single, composite, covering, or partial) with a before-and-after measurement, and I know what it costs.
 - ☐ I can take a table to 3NF and name the dependency each step removes.
 - ☐ I state the grain of every table before joining it, and test it with a query.
 - ☐ I can design a star schema and load a type 2 dimension with a point-in-time join.
 - ☐ I can explain when to denormalize and how to keep a summary honest.
 - ☐ I never change a shared database by hand; I write a numbered, reviewed migration, and I use `expand` and `contract` for renames.
-

Recap

- A **recursive CTE** has an **anchor** and a **recursive part** joined by `UNION ALL`; it repeats until a pass finds nothing new. Use it for org charts, **bills of materials** (multiply quantities down the tree), and **where-used** queries. Guard against loops with a depth limit, a path, or `CYCLE`.
- **Frames**: `ROWS` counts rows, `GROUPS` counts peer groups, `RANGE` measures values (with intervals for calendar windows). `EXCLUDE` removes the current row or its peers; the `WINDOW` clause names a window for reuse. `PERCENTILE_CONT` gives medians; `NTILE` gives equal-count bands.
- Tables live in **pages**. A **sequential scan** reads them all; a **B-tree index** finds rows through sorted keys. `EXPLAIN` shows the plan and estimates; `EXPLAIN ANALYZE` runs it and measures.
- Index **foreign keys** you join on (PostgreSQL doesn't do it for you), keep filters **sargable**, order **composite** indexes equality-first, and use **covering** and **partial** indexes for important, frequent queries. Every index slows writes.
- **Normalization**: **1NF** one value per cell; **2NF** no partial dependencies; **3NF** no transitive dependencies. It removes update, insert, and delete anomalies.
- **Grain**: say it, test it, never join different grains and then sum, and aggregate to a common grain first.
- **Dimensional modeling**: a **fact table** at a declared grain, surrounded by **dimensions** with **surrogate keys**, including a **date dimension**. Prefer a **star** to a **snowflake**. Facts come as transactions, periodic snapshots, or accumulating snapshots.

- **Slowly changing dimensions: type 1** overwrites, **type 2** adds a version with validity dates, **type 3** keeps a previous value. Join type 2 dimensions on the surrogate key or with a **point-in-time join**.
- **Denormalize** only for a measured problem, from a normalized source, with an automated refresh and visible staleness. **Materialized views** trade freshness for speed.
- **Migrations** are numbered, reviewed, never-edited scripts recorded in a history table. Change live structures with **expand and contract**.

Practice exercises

Use `riverstone_2025` (with this chapter's companion tables and the `dw` schema from section 28.9) unless an exercise says otherwise. Exercises 9 and 15 continue in `riverstone_lab`. Predict the shape of each result before running it.

Warm-up

1. Using a recursive CTE, list everyone who reports to Ramesh Patil, directly or indirectly, with their level *relative to him* (his direct reports are level 1).
2. How many people are at each level of the whole organization? Check that the levels add up to 35.
3. State the grain of `lead_stage_history` in one sentence, and write a query that tests it.
4. Find any staff whose `manager_id` points to a staff member who doesn't exist. (A correct query can return no rows. Explain how you know it would catch an orphan.)

Core

5. Produce the whole company as an indented org chart, each manager followed by their team, using PostgreSQL's `SEARCH DEPTH FIRST BY` clause.
6. For the Storage Box 25L, list each bought material, its total quantity per box, and its cost per box. Check that the costs add up to the material cost in section 28.2.
7. Purchasing plans to make 500 Garden Chairs and 1,000 Storage Box 10L next month. How many kilograms of polypropylene granules are needed?
8. For each day in December 2025 that had orders, show that day's number of orders and the number of orders in the 30 days up to and including that day.
9. (Lab.) In section 28.7, `orders_3nf` still stores the sales rep's name. Create `sales_reps_3nf` (with a surrogate key) and `orders_3nf_v2` that stores only the rep's key, then show that orders per rep are unchanged.
10. Using the star schema, show 2025 net revenue by customer city **as it was** and **as it is** for the cities where the two differ.

Stretch

11. Build a monthly sales-versus-target report from the star schema: one row per month, with revenue, target, and percentage of target. Explain in one sentence why you aggregated before joining.

- l2. Show the median order value for each customer segment (as it was) using `PERCENTILE_CONT`, and compare it with the mean.
- l3. Here's a staff extract with a data-entry error. Use `CYCLE` to find the loop, and list the ids that are in it: `(10, NULL), (11, 10), (12, 14), (13, 12), (14, 13), (15, 11)`, as `(staff_id, manager_id)`.
- l4. For each month of 2025, show revenue and the average revenue of the **other** months in the same half-year (January–June or July–December).
- l5. (Lab.) Continue section 28.10's type 2 dimension. A new extract on 1 May 2026 changes Green Leaf Hotels' segment from Hospitality to Retail, and nothing else. Apply it with the same two steps, then show every version of every customer.

Think about it (no SQL needed)

- l6. A query plan shows `Seq Scan on orders ... Filter: (status = 'Delivered') ... Rows Removed by Filter: 25235`, on a table where 96% of orders are delivered. A colleague wants to add an index on `status`. What do you tell them?
- l7. Rewrite each condition so it can use a plain index on its column, or explain why it can't be done: (a) `WHERE DATE(created_at) = '2025-12-15'` on a timestamp column; (b) `WHERE order_date + INTERVAL '30 days' > DATE '2025-12-31'`; (c) `WHERE LOWER(email) = 'rakesh@sharmahardware.example'`; (d) `WHERE customer_name LIKE '%Traders'`.
- l8. Vikram asks for the customer's segment to be copied onto every row of `orders` "so reports don't need a join". Give two risks and a better alternative.
- l9. For each attribute, choose SCD type 1, 2, or 3, with a reason: (a) customer phone number; (b) customer's sales territory, used for commission reports; (c) product category, after a one-off reorganization that management wants to compare side by side for a year; (d) a misspelled customer name.
- l20. Review this migration for a 700,000-row, busy `orders` table and list what's wrong: `ALTER TABLE orders RENAME COLUMN status TO order_status; ALTER TABLE orders ADD COLUMN channel VARCHAR(20) NOT NULL; CREATE INDEX idx_orders_channel ON orders (channel);`

MySQL track (optional)

21. (MySQL.) Write exercise 2 (people per level) in MySQL.
22. (MySQL.) Write the where-used query for the fastener pack (part 306) in MySQL: which products use it, and how many packs per unit?

Key terms

recursive CTE · anchor · recursive part · hierarchy · bill of materials (BOM) · exploding a BOM · where-used query · `CYCLE` clause · frame type · `GROUPS` frame · peer group · `EXCLUDE` · `WINDOW` clause · ordered-set aggregate · `PERCENTILE_CONT` · `NTILE` · page · sequential scan · index · B-tree · leaf page · statistics · query plan · `EXPLAIN` · `EXPLAIN ANALYZE` · cost · heap · index scan · bitmap scan · index-only scan · nested loop join · hash join · merge join · selective · sargable · expression index · composite

index · covering index · partial index · normal form · first normal form (1NF) · update anomaly · insert anomaly · delete anomaly · functional dependency · partial dependency · second normal form (2NF) · transitive dependency · third normal form (3NF) · Boyce–Codd normal form (BCNF) · surrogate key · natural (business) key · grain · dimensional modeling · fact · measure · dimension · fact table · dimension table · star schema · snowflake schema · date dimension · transaction fact table · periodic snapshot · accumulating snapshot · conformed dimension · slowly changing dimension (SCD) · type 1 · type 2 · type 3 · MERGE · point-in-time join · denormalization · materialized view · staleness · migration · drift · schema history table · checksum · repeatable migration · idempotent · expand and contract

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 29, Python as Software, Not Scripts**, turns scripts like `migrate.py` and `generate_riverstone_perf.py` into tested, packaged code.
- **Chapter 32, Analytics Engineering with dbt**, rebuilds this chapter's star schema as dbt models with tests, documentation, and snapshots for type 2 history, replacing hand-written load scripts.
- **Chapter 16, Business Intelligence with Power BI**, reads a star schema like `dw` as its data model.
- **Chapter 47, Data Quality, Observability & Contracts**, turns grain tests and reconciliations into automated daily checks.
- **Chapter 49, Storage, Warehouses & Lakehouses**, takes dimensional models, partitioning, and materialized views to cloud warehouses and columnar storage.
- **Chapter 62, Data Architecture Patterns**, compares warehouse modeling approaches at the architect's level.
- **Chapter 71, SQL Question Bank**, has interview questions on recursive CTEs, indexes, query plans, normalization, grain, and slowly changing dimensions; **Chapter 77** covers data modeling in data engineering and system design interviews.

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. Start the anchor at Ramesh Patil's direct reports, not at Ramesh himself:

```

WITH RECURSIVE team AS (
  SELECT staff_id, staff_name, job_title, 1 AS level
  FROM staff
  WHERE manager_id = (SELECT staff_id FROM staff WHERE staff_name = 'Ramesh Patil')
  UNION ALL
  SELECT s.staff_id, s.staff_name, s.job_title, t.level + 1
  FROM staff AS s
  JOIN team AS t ON s.manager_id = t.staff_id
)
SELECT level, staff_name, job_title
FROM team
ORDER BY level, staff_id;

```

level	staff_name	job_title
1	Ajay Kumar	Shift Supervisor
1	Farhan Ali	Quality Inspector
2	Gopal Sahu	Machine Operator
2	Sunita Pawar	Machine Operator

(4 rows)

Four people, matching Ramesh Patil's `people_below` of 4 in section 28.2. ✓ A common mistake is to anchor on Ramesh himself and forget to exclude him, which returns five rows.

2.

```

WITH RECURSIVE org AS (
  SELECT staff_id, 1 AS level FROM staff WHERE manager_id IS NULL
  UNION ALL
  SELECT s.staff_id, o.level + 1
  FROM staff AS s
  JOIN org AS o ON s.manager_id = o.staff_id
)
SELECT level, COUNT(*) AS people
FROM org
GROUP BY level
ORDER BY level;

```

level	people
1	1
2	8
3	10
4	13
5	3

(5 rows)

$1 + 8 + 10 + 13 + 3 = 35$. ✓

3. Grain: *one lead entering one stage*. If that's true, `(lead_id, stage)` is unique:

```

SELECT lead_id, stage, COUNT(*) AS copies
FROM lead_stage_history
GROUP BY lead_id, stage
HAVING COUNT(*) > 1;

```

```

lead_id | stage | copies
-----+-----+-----
(0 rows)

```

No rows: the grain holds. (The table's primary key already enforces it, but the test still belongs in a data-quality check, because keys are often missing from warehouse copies.)

4.

```

SELECT s.staff_id, s.staff_name, s.manager_id
FROM staff AS s
LEFT JOIN staff AS m ON m.staff_id = s.manager_id
WHERE s.manager_id IS NOT NULL
      AND m.staff_id IS NULL;

```

```

staff_id | staff_name | manager_id
-----+-----+-----
(0 rows)

```

No orphans. The query would catch one because the `LEFT JOIN` keeps every person, and a `manager_id` with no matching staff row leaves `m.staff_id` `NULL`. The `manager_id IS NOT NULL` condition stops the managing director, who legitimately has no manager, from being reported. (The foreign key on `manager_id` prevents orphans in this table; copies loaded into spreadsheets or warehouses often lack it.)

5.

```

WITH RECURSIVE org AS (
  SELECT staff_id, staff_name, manager_id, 1 AS level
  FROM staff
  WHERE manager_id IS NULL
  UNION ALL
  SELECT s.staff_id, s.staff_name, s.manager_id, o.level + 1
  FROM staff AS s
  JOIN org AS o ON s.manager_id = o.staff_id
)
SEARCH DEPTH FIRST BY staff_id SET visit_order
SELECT repeat('  ', level - 1) || staff_name AS org_chart
FROM org
ORDER BY visit_order;

```

```

org_chart
-----
Arvind Kapoor
  Anita Rao
    Vikram Singh
      Neha Kulkarni
      Rahul Mehta
    Farah Khan
    Meera Iyer
    Arjun Nair
      Divya Krishnan
      Vivek Chandran
      Sneha Pillai
    Pooja Desai
      Nisha Bhatt
      Aditya Verma
      Rohit Kamat
    Sandeep Gill
      Karan Ahuja
      Ritu Bansal
  Suresh Menon
    Priya Nambiar
  Harpreet Sethi
    Ramesh Patil
      Ajay Kumar
      Gopal Sahu
      Sunita Pawar
      Farhan Ali
    Kiran Bhosale
      Swati Joshi
      Farid Shaikh
  Joseph D'Souza
  Mahesh Yadav
    Mohan Das
  Lakshmi Reddy
  Zoya Mirza
  Tenzin Dorji
(35 rows)

```

SEARCH DEPTH FIRST BY `staff_id` adds a hidden column, here named `visit_order`, that sorts each person immediately after their manager, with siblings in `staff_id` order. SEARCH BREADTH FIRST would sort level by level instead, like section 28.2's first query.

6.

```

WITH RECURSIVE explode AS (
  SELECT b.child_part_id, CAST(b.quantity AS NUMERIC) AS qty
  FROM bom_lines AS b
  WHERE b.parent_part_id = 102
  UNION ALL
  SELECT b.child_part_id, e.qty * b.quantity
  FROM explode AS e
  JOIN bom_lines AS b ON b.parent_part_id = e.child_part_id
)
SELECT p.part_name,
       CAST(SUM(e.qty) AS NUMERIC(8,3)) AS qty_per_box,
       p.unit,
       CAST(SUM(e.qty) * p.material_cost AS NUMERIC(10,2)) AS cost_per_box
FROM explode AS e
JOIN parts AS p ON p.part_id = e.child_part_id
WHERE p.part_type = 'Material'
GROUP BY p.part_id, p.part_name, p.unit, p.material_cost
ORDER BY p.part_id;

```

part_name	qty_per_box	unit	cost_per_box
Polypropylene granules	1.500	kg	165.00
Color masterbatch	0.055	kg	14.30
Product label	1.000	each	2.00
Shipping carton	1.000	each	18.00

(4 rows)

₹165.00 + ₹14.30 + ₹2.00 + ₹18.00 = ₹199.30, matching section 28.2. ✓ The granules are 1.1 kg in the body plus 0.4 kg in the lid, so group by part before multiplying by cost.

7.

```

WITH RECURSIVE plan (product_id, units) AS (
  VALUES (106, 500), (101, 1000)
),
explode AS (
  SELECT b.child_part_id, CAST(b.quantity * p.units AS NUMERIC) AS qty
  FROM plan AS p
  JOIN bom_lines AS b ON b.parent_part_id = p.product_id
  UNION ALL
  SELECT b.child_part_id, e.qty * b.quantity
  FROM explode AS e
  JOIN bom_lines AS b ON b.parent_part_id = e.child_part_id
)
SELECT CAST(SUM(qty) AS NUMERIC(10,1)) AS kg_polypropylene
FROM explode
WHERE child_part_id = 301;

```

kg_polypropylene

1850.0

(1 row)

Hand-check: a chair needs 2.2 kg ($500 \times 2.2 = 1,100$ kg); a 10-litre box needs $0.55 + 0.20 = 0.75$ kg ($1,000 \times 0.75 = 750$ kg). Total 1,850 kg. ✓ Putting the production plan into a `VALUES` list at the start makes the anchor multiply by planned units, so the rest of the query is unchanged. (The first CTE isn't recursive; `WITH RECURSIVE` is written once for the whole list.)

8. Count orders per day first (the grain is one day), then use a `RANGE` frame. Starting the daily counts 29 days before December makes the first December rows see a full 30 days:

```
WITH daily AS (  
    SELECT order_date, COUNT(DISTINCT order_id) AS orders  
    FROM sales_lines  
    WHERE order_date >= DATE '2025-11-02'  
    GROUP BY order_date  
)  
,  
rolling AS (  
    SELECT order_date,  
           orders,  
           SUM(orders) OVER (ORDER BY order_date  
                             RANGE BETWEEN INTERVAL '29 days' PRECEDING AND CURRENT ROW) AS  
orders_last_30_days  
    FROM daily  
)  
SELECT *  
FROM rolling  
WHERE order_date >= DATE '2025-12-01'  
ORDER BY order_date;
```

order_date	orders	orders_last_30_days
2025-12-01	2	23
2025-12-04	1	23
2025-12-05	1	23
2025-12-08	1	23
2025-12-10	2	22
2025-12-12	1	20
2025-12-13	1	21
2025-12-14	1	22
2025-12-16	3	22
2025-12-17	1	23
2025-12-18	1	23
2025-12-19	1	23
2025-12-21	1	24
2025-12-23	1	22

(14 rows)

`INTERVAL '29 days' PRECEDING AND CURRENT ROW` covers 30 days including today. The common wrong answer is `ROWS BETWEEN 29 PRECEDING AND CURRENT ROW`, which covers 30 *order days*, more than a month in this data.

9.

```

CREATE TABLE sales_reps_3nf (
    sales_rep_id INTEGER PRIMARY KEY,
    sales_rep    VARCHAR(100) NOT NULL UNIQUE
);

INSERT INTO sales_reps_3nf
SELECT ROW_NUMBER() OVER (ORDER BY sales_rep), sales_rep
FROM (SELECT DISTINCT sales_rep FROM orders_3nf) AS r;

CREATE TABLE orders_3nf_v2 AS
SELECT o.order_id, o.order_date, o.customer_id, r.sales_rep_id
FROM orders_3nf AS o
JOIN sales_reps_3nf AS r ON r.sales_rep = o.sales_rep;

SELECT 'before' AS version, sales_rep, COUNT(*) AS orders
FROM orders_3nf
GROUP BY sales_rep
UNION ALL
SELECT 'after', r.sales_rep, COUNT(*)
FROM orders_3nf_v2 AS o
JOIN sales_reps_3nf AS r ON r.sales_rep_id = o.sales_rep_id
GROUP BY r.sales_rep
ORDER BY sales_rep, version DESC;

```

version	sales_rep	orders
before	Farah Khan	2
after	Farah Khan	2
before	Neha Kulkarni	3
after	Neha Kulkarni	3
before	Rahul Mehta	1
after	Rahul Mehta	1

(6 rows)

Same counts before and after (2 + 1 + 3 = 6 orders). ✓ The rep's name now lives in one row, so a corrected spelling fixes every order at once.

10.

```

WITH was AS (
    SELECT c.city, SUM(f.net_revenue) AS revenue
    FROM dw.fact_sales_line AS f
    JOIN dw.dim_customer AS c ON c.customer_key = f.customer_key
    GROUP BY c.city
),
is_now AS (
    SELECT cur.city, SUM(f.net_revenue) AS revenue
    FROM dw.fact_sales_line AS f
    JOIN dw.dim_customer AS c ON c.customer_key = f.customer_key
    JOIN dw.dim_customer AS cur ON cur.customer_id = c.customer_id AND cur.is_current
    GROUP BY cur.city
)
SELECT COALESCE(w.city, i.city) AS city,
       ROUND(w.revenue)       AS revenue_as_it_was,
       ROUND(i.revenue)       AS revenue_as_it_is
FROM was AS w
FULL JOIN is_now AS i ON i.city = w.city
WHERE w.revenue IS DISTINCT FROM i.revenue
ORDER BY city;

```

city	revenue_as_it_was	revenue_as_it_is
Mumbai	928760	1080800
Thane	152040	

(2 rows)

Only Metro Mart moved city, so only Mumbai and Thane differ: Metro Mart's orders from January to June 2025 count under Thane as they were. The `FULL JOIN` matters: Thane has no revenue "as it is" (no current customer is in Thane), and an inner join would drop it.

11.

```
WITH monthly AS (
    SELECT d.month_start, SUM(f.net_revenue) AS revenue
    FROM dw.fact_sales_line AS f
    JOIN dw.dim_date AS d ON d.date_key = f.date_key
    GROUP BY d.month_start
)
SELECT m.month_start,
       ROUND(m.revenue) AS revenue,
       ROUND(t.target_revenue) AS target,
       ROUND(100 * m.revenue / t.target_revenue, 1) AS pct_of_target
FROM monthly AS m
JOIN sales_targets AS t ON t.target_month = m.month_start
ORDER BY m.month_start;
```

month_start	revenue	target	pct_of_target
2025-01-01	202640	300000	67.5
2025-02-01	253664	300000	84.6
2025-03-01	278008	320000	86.9
2025-04-01	210282	320000	65.7
2025-05-01	329359	320000	102.9
2025-06-01	186928	300000	62.3
2025-07-01	232692	280000	83.1
2025-08-01	329282	320000	102.9
2025-09-01	558315	380000	146.9
2025-10-01	681071	520000	131.0
2025-11-01	633408	500000	126.7
2025-12-01	439824	380000	115.7

(12 rows)

The fact table's grain is one order line and the target's grain is one month; aggregating the facts to months first makes the join one-to-one, so the target is counted once (section 28.8). January's 67.5% matches section 28.8. ✓

12.

```

WITH order_values AS (
    SELECT c.segment, f.order_id, SUM(f.net_revenue) AS order_value
    FROM dw.fact_sales_line AS f
    JOIN dw.dim_customer AS c ON c.customer_key = f.customer_key
    GROUP BY c.segment, f.order_id
)
SELECT segment,
       COUNT(*) AS orders,
       ROUND(AVG(order_value)) AS mean_value,
       ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY order_value)::NUMERIC) AS median_value
FROM order_values
GROUP BY segment
ORDER BY segment;

```

segment	orders	mean_value	median_value
Hospitality	59	19390	17589
Retail	62	25381	22649
Wholesale	52	31112	29199

(3 rows)

The order counts add up to 173. ✓ In every segment the mean is above the median, so each segment has a few large orders pulling its average up. `PERCENTILE_CONT` is an aggregate, so it works with `GROUP BY` like `AVG`.

13.

```

WITH RECURSIVE extract_rows (staff_id, manager_id) AS (
    VALUES (10, NULL), (11, 10), (12, 14), (13, 12), (14, 13), (15, 11)
),
walk AS (
    SELECT staff_id, manager_id, staff_id AS started_at
    FROM extract_rows
    UNION ALL
    SELECT e.staff_id, e.manager_id, w.started_at
    FROM extract_rows AS e
    JOIN walk AS w ON e.staff_id = w.manager_id
)
CYCLE staff_id SET is_loop USING visited
SELECT DISTINCT started_at AS staff_in_loop
FROM walk
WHERE is_loop AND staff_id = started_at
ORDER BY started_at;

```

staff_in_loop
12
13
14

(3 rows)

The walk starts at every person and goes up the chain. People 10, 11, and 15 reach the top (10 has no manager). Starting from 12, 13, or 14, the walk comes back to its starting person, so those three form the loop (12 → 14 → 13 → 12). Filtering on `staff_id = started_at` keeps only people who return to themselves; someone *below* a loop would reach it without returning to their own id.

14.

```

WITH monthly AS (
    SELECT DATE_TRUNC('month', order_date)::date AS month,
           SUM(net_revenue) AS revenue
    FROM sales_lines
    GROUP BY 1
)
SELECT month,
       ROUND(revenue) AS revenue,
       ROUND(AVG(revenue) OVER (
           PARTITION BY EXTRACT(MONTH FROM month) <= 6
           ORDER BY month
           ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
           EXCLUDE CURRENT ROW)) AS avg_other_months_in_half
FROM monthly
ORDER BY month;

```

month	revenue	avg_other_months_in_half
2025-01-01	202640	251648
2025-02-01	253664	241443
2025-03-01	278008	236574
2025-04-01	210282	250120
2025-05-01	329359	226304
2025-06-01	186928	254790
2025-07-01	232692	528380
2025-08-01	329282	509062
2025-09-01	558315	463255
2025-10-01	681071	438704
2025-11-01	633408	448237
2025-12-01	439824	486954

(12 rows)

Partitioning by the true/false expression `EXTRACT(MONTH FROM month) <= 6` makes two halves. Each month is compared with the five others in its half.

15.

```

DELETE FROM customer_extract;
INSERT INTO customer_extract VALUES
(1, 'Sharma Hardware', 'Thane', 'Retail'),
(3, 'Green Leaf Hotels', 'Pune', 'Retail'),
(4, 'Coastal Foods', 'Chennai', 'Wholesale');

BEGIN;

UPDATE dim_customer_t2 AS d
SET valid_to = DATE '2026-04-30',
    is_current = FALSE
FROM customer_extract AS x
WHERE d.customer_id = x.customer_id
    AND d.is_current
    AND (d.city IS DISTINCT FROM x.city OR d.segment IS DISTINCT FROM x.segment);

INSERT INTO dim_customer_t2 (customer_id, customer, city, segment, valid_from)
SELECT x.customer_id, x.customer, x.city, x.segment, DATE '2026-05-01'
FROM customer_extract AS x
LEFT JOIN dim_customer_t2 AS d
ON d.customer_id = x.customer_id AND d.is_current
WHERE d.customer_id IS NULL;

COMMIT;

SELECT customer_key, customer_id, customer, city, segment, valid_from, valid_to, is_current
FROM dim_customer_t2
ORDER BY customer_id, valid_from;

```

	customer_key	customer_id	customer	city	segment	valid_from	valid_to	is_current
f	1	1	Sharma Hardware	Mumbai	Retail	2024-11-04	2026-03-31	
t	3	1	Sharma Hardware	Thane	Retail	2026-04-01	9999-12-31	
f	2	3	Green Leaf Hotels	Pune	Hospitality	2024-09-08	2026-04-30	
t	5	3	Green Leaf Hotels	Pune	Retail	2026-05-01	9999-12-31	
t	4	4	Coastal Foods	Chennai	Wholesale	2026-04-01	9999-12-31	

(5 rows)

Only Green Leaf Hotels gets a new version. Sharma Hardware and Coastal Foods are unchanged, so step 1 leaves their current rows open and step 2 finds they already have one. Running the same two steps a second time would change nothing, which is how you check a load is idempotent. (The date `2026-04-30` closes the version on the day before the new one starts; with this chapter's half-open joins, `valid_to` could equally store the start date of the next version, as `dw.dim_customer` does. Pick one convention and use it everywhere.)

16. Don't add it. The filter keeps 96% of rows, so an index on `status` would be slower than scanning: the planner would ignore it, as it did for `'Delivered'` in section 28.5, and every insert and status change would pay to maintain it. If a query really needs the rare statuses (pending or cancelled), a **partial index** on those rows is the right tool. And if the delivered query is slow, the fix is elsewhere: a summary table, or not reading every delivered order.

17. (a) `WHERE created_at >= '2025-12-15' AND created_at < '2025-12-16'` . The half-open range also includes times after midnight, which `BETWEEN '2025-12-15' AND '2025-12-15'` would miss. (b) Move the arithmetic to the constant side: `WHERE order_date > DATE '2025-12-31' - INTERVAL '30 days'` , which is `order_date > '2025-12-01'` . (c) Store emails in one case when they're saved and compare directly, or create an expression index on `LOWER(email)` and keep the query as written. (d) A plain B-tree index can't find text by its *ending*. Options: a reversed-text expression index, a trigram index (PostgreSQL's `pg_trgm` extension), or a separate `business_type` column. The common wrong answer is "add an index on `customer_name` ", which the planner would not use for a leading % .

18. Risks: (1) the copies disagree when a customer's segment changes and not every order is updated, an update anomaly (section 28.7); (2) it's unclear whether an order should show the segment at the time of the order or today's, and the copy silently picks one. Better alternatives: a star schema with a type 2 customer dimension (both views available, section 28.10), or a view that joins orders to customers so reports don't write the join themselves. If the join is truly too slow, a materialized view refreshed on a schedule keeps the copy derived and rebuildable.

19. (a) Type 1. Nobody reports sales by old phone numbers; the new one replaces it. (b) Type 2. Commission is calculated on who owned the customer when the sale happened, so history must be kept. (c) Type 3. A `previous_category` column lets reports show this year's sales under both the old and new categories side by side, which is exactly the request. (If they later wanted the full history, type 2 would be needed.) (d) Type 1. It's a correction: the misspelled name was never true, so there's no history to keep.

20. Five problems. (1) The rename breaks every report, view, and application that reads `status` , at the moment it runs: use `expand` and `contract`. (2) `channel VARCHAR(20) NOT NULL` with no default fails on a table with rows (*column "channel" ... contains null values*): add a default, or add it nullable, backfill, then set `NOT NULL` . (3) A plain `CREATE INDEX` blocks writes on a busy table while it builds: use `CREATE INDEX CONCURRENTLY` , in its own migration outside a transaction. (4) Three unrelated changes in one migration: split them. (5) An index on `channel` before anyone has measured a query that needs it: add it only for a known query, preferably with a before-and-after plan.

21.

```
WITH RECURSIVE org AS (  
    SELECT staff_id, 1 AS level FROM staff WHERE manager_id IS NULL  
    UNION ALL  
    SELECT s.staff_id, o.level + 1  
    FROM staff AS s  
    JOIN org AS o ON s.manager_id = o.staff_id  
)  
SELECT level, COUNT(*) AS people  
FROM org  
GROUP BY level  
ORDER BY level;
```

level	people
1	1
2	8
3	10
4	13
5	3

Identical to PostgreSQL's result in exercise 2. No `CAST` is needed, because `level` is a number and doesn't grow in size.

22.

```
WITH RECURSIVE uses AS (
    SELECT b.parent_part_id, CAST(b.quantity AS DECIMAL(12,3)) AS qty
    FROM bom_lines AS b
    WHERE b.child_part_id = 306
    UNION ALL
    SELECT b.parent_part_id, CAST(u.qty * b.quantity AS DECIMAL(12,3))
    FROM uses AS u
    JOIN bom_lines AS b ON b.child_part_id = u.parent_part_id
)
SELECT p.part_name, SUM(u.qty) AS packs_per_unit
FROM uses AS u
JOIN parts AS p ON p.part_id = u.parent_part_id
WHERE p.part_type = 'Product'
GROUP BY p.part_name;
```

part_name	packs_per_unit
Garden Chair	3.000

Three packs per Garden Chair: one in the frame and one in each of the two leg assemblies (Figure 28.2). ✓ In MySQL, the `CAST` in both parts keeps the column's type fixed, so the multiplied quantities aren't cut to the anchor's precision.